

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI



BÁO CÁO THỰC TẬP

CHỦ ĐỀ: Chatbot hỗ trợ hệ thống SCP

Người hướng dẫn: Đặng Quang Dũng
Hồ Quang Chung

Nhóm sinh viên thực hiện:

Họ và tên	MSV
Mai Thế Cường	23001504
Phạm Trung Hiếu	23001876
Lê Xuân Lộc	23001533

Hà Nội, năm 2026

Mục lục

1	MỞ ĐẦU	5
1.1	Bối cảnh nghiên cứu và Đặt vấn đề	5
1.2	Mục tiêu nghiên cứu của hệ thống	5
1.3	Tính cách ly và Các rào chắn an toàn (Guardrails)	6
2	CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC NỀN TẢNG	6
2.1	Phân rã tác vụ trong Mô hình ngôn ngữ lớn (Task Decomposition)	6
2.2	Kiến trúc Dữ liệu Phân tầng (Layered Data Architecture)	7
2.3	Cơ chế Truy hồi lai (Hybrid Information Retrieval)	7
2.4	Thuật toán Dung hợp kết quả (Reciprocal Rank Fusion - RRF)	7
3	KIẾN TRÚC ĐIỀU PHỐI TRUNG TÂM VÀ BỘ ĐỊNH TUYẾN	8
3.1	Phân loại truy vấn và Trích xuất thực thể bằng LLM (light)	8
3.2	Rào chắn định tuyến dựa trên ngữ cảnh giao diện (UI Routing Guardrail)	9
3.3	Cơ chế Thực thi song song đa luồng (Parallel Tool Execution)	10
4	PHÂN TÍCH LUỒNG US01 - TRỢ LÝ HƯỚNG DẪN THAO TÁC	10
4.1	Đặc tả luồng xử lý và Ranh giới nghiệp vụ	11
4.2	Cơ chế mã hóa ngữ nghĩa (Embedding) với BGE-M3	12
4.3	Thuật toán tìm kiếm lai (Hybrid Retrieval) và Lọc ứng viên	13
4.4	Chiến lược kết xuất trực tiếp (Direct Render Bypass) tối ưu độ trễ	14
4.5	Cơ chế tổng hợp đa bước và Ràng buộc kiểm tra (Grounding Check)	15
5	PHÂN TÍCH LUỒNG US 2 - Giải thích Hệ thống	16
5.1	Mục đích và phạm vi US02	16
5.2	Ranh giới với US01 và US03	17
5.3	Kiến trúc hệ thống	18
5.3.1	Vị trí US02 trong tổng thể Assistant Core	18

5.3.2	Corpus runtime — 5 nhóm dữ liệu	18
5.3.3	Sơ đồ luồng xử lý (Agent flow)	19
5.4	Cơ chế định tuyến vào US02	20
5.4.1	Vai trò tool selector (LLM <i>light</i>)	20
5.4.2	Rào chắn 5 điều kiện	21
5.4.3	Vì sao không tin screen_id/module từ browser	21
5.5	Cơ chế Retrieval	22
5.5.1	Ba kênh: Exact, BM25, BGE-M3	22
5.5.2	Công thức và tham số RRF	23
5.5.3	Bảng cấu hình hiện tại	23
5.5.4	Cơ chế fallback khi vector lỗi (degraded_channels)	24
5.6	Cơ chế kiểm citation	24
5.7	Kiểm soát chất lượng tài liệu (Coverage)	26
5.7.1	ui-inventory.json và gap-report.json	26
5.7.2	Coverage là quality gate, không phải đảm bảo độ chính xác	27
5.8	Vận hành và chi phí	27
5.8.1	Số lời gọi LLM mỗi lượt	28
5.8.2	Audit và feedback	29
6	PHÂN TÍCH LUỒNG US03 - TRUY VẤN SỐ LIỆU THỰC TẾ	30
6.1	Mục đích, phạm vi và đặc trưng của US03	30
6.2	Kiến trúc dữ liệu phục vụ US03	35
6.2.1	Vị trí US03 trong tổng thể Assistant Core	35
6.2.2	Tầng ngữ nghĩa Semantic Layer	37
6.2.3	Binding Pack và phạm vi dữ liệu được phê duyệt	38
6.2.4	Metric Registry và hợp đồng nghiệp vụ	40
6.3	Kiến trúc tổng thể của luồng xử lý US03	43
6.3.1	Sơ đồ End-to-End	44
6.3.2	Các giai đoạn xử lý chính	46

6.3.3	Vai trò của LLM và các thành phần tất định	51
6.4	Tiếp nhận và phân tích truy vấn	53
6.4.1	Định tuyến truy vấn vào query_data	53
6.4.2	Nhận diện Metric và ngữ cảnh nghiệp vụ	56
6.5	Lập kế hoạch truy vấn dữ liệu	60
6.5.1	Lựa chọn Semantic View	61
6.5.2	Sinh kế hoạch truy vấn SqlPlan	63
6.6	Kiểm tra tính đúng đắn của truy vấn	66
6.6.1	Kiểm tra Hợp đồng nghiệp vụ	66
6.6.2	Cơ chế Self-Correction và Retry	71
6.7	Cơ chế bảo vệ và thực thi SQL	74
6.7.1	SQL Guardrail dựa trên AST	75
6.7.2	Kiểm soát Fan-out và phép tổng hợp	79
6.7.3	Kiểm soát chi phí bằng EXPLAIN	81
6.7.4	Thực thi Read-only	83

Danh sách hình vẽ

1	Sơ đồ khối luồng xử lý và truy hồi tài liệu thao tác (US01 Pipeline)	11
2	Sơ đồ luồng xử lý (Agent flow) của US02	19
3	Cơ chế kiểm citation: đối chiếu câu trả lời của LLM với evidence pack trước khi render	25
4	Sơ đồ End-to-End của luồng xử lý US03	45

Danh sách bảng

1	Ranh giới định tuyến giữa US01, US02, US03 theo dạng câu hỏi	17
2	Năm nhóm dữ liệu trong corpus runtime của US02	18
3	Vai trò của ba kênh trong Hybrid Retrieval	22
4	Cấu hình hiện tại của tầng Retrieval — US02	23
5	Ba lời gọi LLM trong một lượt xử lý US02	28
6	Các thành phần chính của Binding Pack phục vụ US03	39
7	Phạm vi dữ liệu hiện tại của US03	40
8	Vai trò của LLM và thành phần tất định trong US03	52
9	Phân tích ngữ cảnh nghiệp vụ của một truy vấn US03	58
10	Phân chia trách nhiệm trong bước lựa chọn Semantic View	62
11	Các kiểm tra chính trong Business Contract Validation	71
12	Các lớp kiểm tra SQL trước khi thực thi	83

Phần 1. MỞ ĐẦU

1.1 Bối cảnh nghiên cứu và Đặt vấn đề

Các hệ thống quản lý chuỗi cung ứng và kế hoạch (Supply Chain Planning - SCP) quy mô doanh nghiệp thường mang kiến trúc dữ liệu và logic nghiệp vụ rất phức tạp. Trong quá trình vận hành, khoảng cách ngữ nghĩa (semantic gap) giữa giao diện người dùng và cấu trúc cơ sở dữ liệu bên dưới là rất lớn. Người dùng gặp khó khăn trong việc tra cứu luồng thao tác, hiểu ý nghĩa các trường dữ liệu, hoặc truy xuất số liệu tổng hợp.

Việc tích hợp các Mô hình ngôn ngữ lớn (LLM) để giải quyết vấn đề này mở ra một hướng tiếp cận tiềm năng mang tên trợ lý ảo (Chatbot). Tuy nhiên, đứng từ góc độ kỹ thuật hệ thống (systems engineering), việc để LLM tương tác trực tiếp với hệ thống lõi tiềm ẩn các rủi ro lớn về tính phi định định (non-determinism), an toàn dữ liệu, và rò rỉ thông tin nghiệp vụ. Do đó, bài toán đặt ra là làm thế nào để thiết kế một kiến trúc trợ lý AI đủ thông minh để hiểu ngữ cảnh người dùng, nhưng phải hoàn toàn tách biệt, có khả năng kiểm soát chặt chẽ luồng thực thi và giới hạn phạm vi truy xuất thông tin.

1.2 Mục tiêu nghiên cứu của hệ thống

Nghiên cứu này tập trung vào việc thiết kế và xây dựng một kiến trúc hệ thống Chatbot (Assistant Core) hoạt động độc lập với phân hệ frontend và backend của SCP. Mục tiêu cốt lõi là tạo ra một bộ điều phối (orchestrator) có khả năng định tuyến các truy vấn ngôn ngữ tự nhiên một cách an toàn và có cấu trúc.

Để giải quyết tính không đồng nhất của truy vấn, hệ thống phân rã bài toán thành ba luồng xử lý chuyên biệt (sub-systems/tools):

- Phân hệ US01 (**search_docs**): Tập trung vào bài toán truy hồi tài liệu (Document Retrieval), phục vụ các truy vấn về hướng dẫn thao tác và quy trình người dùng.
- Phân hệ US02 (**lookup_field**): Vận hành dưới dạng một Agent trích xuất tri thức, làm nhiệm vụ ánh xạ (mapping) giữa nhãn giao diện (UI) với logic backend, rule, công thức và cấu trúc cơ sở dữ liệu.
- Phân hệ US03 (**query_data**): Giải quyết bài toán Text-to-SQL trong môi trường bị giới hạn, cho phép truy vấn số liệu thực tế dựa trên một tầng ngữ nghĩa (semantic view) đã được định nghĩa và phê duyệt nghiêm ngặt.

1.3 Tính cách ly và Các rào chắn an toàn (Guardrails)

Một trong những đóng góp chính của kiến trúc này là việc thiết lập các ranh giới bảo mật cứng (hard boundaries) nhằm cô lập rủi ro từ LLM. Hệ thống tuân thủ các nguyên tắc thiết kế sau:

- Nguyên tắc Read-only và Isolation: Chatbot vận hành song song nhưng không can thiệp, không thay thế, và không có quyền ghi (write) vào cơ sở dữ liệu nghiệp vụ gốc.
- Nguyên tắc Bounded Execution: Quá trình LLM sinh truy vấn hoặc diễn giải dữ liệu không được phép vượt ra ngoài tập hợp các view, cột, metric và quy tắc đã được thiết lập sẵn. Mọi hành vi tự suy diễn ngoài phạm vi đều bị chặn ở cấp độ runtime.
- Kiến trúc Stateless API: Lỗi Assistant Core giao tiếp thông qua một hợp đồng API chuẩn mực. Tính phi trạng thái (stateless) này giúp hệ thống dễ dàng mở rộng (scale) và không bị phụ thuộc vào một nền tảng frontend cụ thể nào.

Phần 2. CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC NỀN TẢNG

2.1 Phân rã tác vụ trong Mô hình ngôn ngữ lớn (Task Decomposition)

Thay vì sử dụng một LLM nguyên khối (monolithic prompt) để xử lý từ đầu đến cuối, hệ thống áp dụng chiến lược phân rã tác vụ (task decomposition). Cách tiếp cận này giúp tối ưu hóa chi phí tính toán (token usage), giảm độ trễ (latency) và tăng tính định định cho hệ thống. LLM được tinh chỉnh và phân vai trò cụ thể như sau:

- Router/Selector (**light**): Hoạt động như một bộ phân loại (classifier), sử dụng các model nhẹ, tốc độ cao để định tuyến truy vấn và trích xuất thực thể (entity extraction) mà không sinh văn bản trực tiếp cho người dùng.
- Generator (**sql_generation**): Được tối ưu hóa chuyên biệt cho bài toán biên dịch ngôn ngữ tự nhiên sang cú pháp SQL có cấu trúc.
- Planner (**us02_planning**): Chịu trách nhiệm lập kế hoạch truy hồi dữ liệu, quyết định các tham số tìm kiếm và xác định nguồn bằng chứng (evidence) cần thiết.
- Synthesizer (**synthesis, us02_synthesis**): Thực hiện tổng hợp thông tin từ tập kết quả truy hồi và sinh câu trả lời ngôn ngữ tự nhiên dựa trên cơ chế bắt buộc trích dẫn (mandatory citation).

Việc thiết kế hệ thống tương thích ngược với chuẩn giao thức OpenAI thông qua OPENAI_API_KEY cho phép kiến trúc linh hoạt thay đổi nhà cung cấp nền tảng lõi (như Gemini, Groq) tùy thuộc vào bài toán tối ưu chi phí và năng lực tính toán.

2.2 Kiến trúc Dữ liệu Phân tầng (Layered Data Architecture)

Để giải quyết mâu thuẫn giữa nhu cầu truy xuất dữ liệu động của LLM và yêu cầu bảo mật của hệ thống SCP, một kiến trúc dữ liệu phân tầng được đề xuất:

- **Tầng Tri thức độc lập (Knowledge Base):** Tách biệt hoàn toàn với CSDL nghiệp vụ. Sử dụng hệ quản trị PostgreSQL tích hợp phần mở rộng pgvector để lưu trữ và lập chỉ mục không gian vector cho tài liệu, corpus ngữ nghĩa, và nhật ký kiểm toán (audit logs).
- **Tầng Ngữ nghĩa (Semantic Layer):** Tạo ra một lớp trừu tượng (abstraction layer) mang tên `assistant` bọc ngoài cơ sở dữ liệu gốc. Lớp này đóng vai trò như một bộ lọc, chỉ bộc lộ (expose) các bảng, cột, phép kết nối (join) và công thức tính toán an toàn cho LLM ở luồng US03, chặn đứng rủi ro khai thác dữ liệu nhạy cảm bằng kỹ thuật prompt injection.

2.3 Cơ chế Truy hồi lai (Hybrid Information Retrieval)

Các bài toán tìm kiếm thông tin trong môi trường doanh nghiệp đòi hỏi độ chính xác tuyệt đối đối với các định danh kỹ thuật, nhưng cũng cần tính linh hoạt với ngôn ngữ tự nhiên. Để giải quyết, hệ thống triển khai cơ chế tìm kiếm lai (Hybrid Retrieval) gồm ba vector thành phần:

- **Cơ chế Khớp chính xác (Exact match):** Được ưu tiên cao nhất, thuật toán này ánh xạ độ chuẩn xác tỷ lệ 1:1 đối với các nhãn UI, bí danh (alias) và tên trường (field) từ metadata của hệ thống.
- **Cơ chế Truy hồi Sparse Vector (BM25):** Dựa trên thống kê tần suất từ vựng, giúp giải quyết sự bất cập các biến thể từ khóa.
- **Cơ chế Truy hồi Dense Vector (BGE-M3):** Vận hành nội bộ trên môi trường CPU nhằm bảo mật luồng dữ liệu và loại bỏ độ trễ gọi API bên ngoài. Mô hình nhúng (embedding) tính toán khoảng cách ngữ nghĩa giữa các vector, giúp nắm bắt ý định người dùng kể cả khi họ dùng từ đồng nghĩa hoặc nhập văn bản không dấu.

Đặc biệt, hệ thống được thiết kế với cơ chế dự phòng (fault tolerance). Nếu thành phần BGE-M3 quá tải hoặc lỗi, hệ thống tự động giảm cấp (degrade) sang sử dụng Exact Match và BM25 thay vì đổ vỡ toàn hệ thống.

2.4 Thuật toán Dung hợp kết quả (Reciprocal Rank Fusion - RRF)

Việc kết hợp kết quả từ các cơ chế truy hồi khác biệt (vốn có thang điểm đánh giá độc lập) đòi hỏi một thuật toán chuẩn hóa phi tham số. Nghiên cứu áp dụng thuật toán Reciprocal Rank Fusion

(RRF) để gộp và tái xếp hạng danh sách bằng chứng. Điểm số RRF của một tài liệu d được mô hình hóa qua phương trình:

$$Score(d) = \sum_c \frac{1}{60 + rank_c(d)} \quad (1)$$

Trong đó:

- c : Tập hợp các hệ thống con tìm kiếm tham gia (Exact, BM25, BGE-M3).
- $rank_c(d)$: Thứ hạng cục bộ của tài liệu d trong hệ thống con c .
- Hằng số làm trơn 60 được sử dụng để giảm thiểu độ lệch chuẩn của các tài liệu có thứ hạng quá chênh lệch giữa các hệ thống con.

Để đáp ứng tính chất nghiêm ngặt của miền nghiệp vụ, hệ thống cấp một trọng số thiên vị (bias weight) tuyệt đối cho các kết quả trả về từ Exact match, ép buộc thuật toán luôn bảo toàn vị trí của các thông tin định danh hệ thống (system identifiers) lên trên các kết quả nội suy ngữ nghĩa.

Phần 3. KIẾN TRÚC ĐIỀU PHỐI TRUNG TÂM VÀ BỘ ĐỊNH TUYẾN

Trong một kiến trúc trợ lý ảo đa nhiệm, nếu để một Mô hình ngôn ngữ lớn (LLM) duy nhất xử lý toàn bộ truy vấn từ đầu đến cuối, hệ thống sẽ đối mặt với các rủi ro nghiêm trọng về ảo giác (hallucination), độ trễ cao và khó kiểm soát quyền truy cập. Để giải quyết vấn đề này, Assistant Core của SCP được thiết kế xoay quanh một Bộ điều phối trung tâm (Orchestrator). Bộ điều phối này đóng vai trò như một màng lọc và định tuyến, đảm bảo mỗi truy vấn được phân rã và chuyển đến đúng phân hệ chuyên trách.

3.1 Phân loại truy vấn và Trích xuất thực thể bằng LLM (**light**)

Cơ chế cốt lõi của Orchestrator là sử dụng một LLM được tối ưu hóa cho tốc độ và khả năng phân tích logic, gọi là tác vụ **light**. Khác với các hệ thống chatbot tạo sinh (generative) thông thường, LLM **light** trong kiến trúc SCP không được phép trực tiếp sinh văn bản trả lời cho người dùng. Nhiệm vụ duy nhất của nó là đọc hiểu truy vấn, trích xuất các thực thể (entity) ban đầu (như tên module, từ khóa) và lựa chọn công cụ (tool) phù hợp.

Quá trình định tuyến này dựa trên việc thiết lập các ranh giới ý nghĩa nghiêm ngặt giữa ba luồng:

- **Luồng Thao tác (US01)**: Được kích hoạt khi bộ điều phối phát hiện các tín hiệu mang tính hành động và quy trình. Ví dụ: "*Làm sao tạo...*", "*Cách gửi...*", "*Quy trình import...*".

- **Luồng Giải thích (US02):** Được ưu tiên cho các truy vấn mang tính khái niệm, khai phá logic ẩn hoặc tìm hiểu công thức. Các dấu hiệu nhận biết bao gồm: *"X là gì?"*, *"X được tính thế nào?"*, *"X liên quan đến Y ra sao?"*.
- **Luồng Số liệu (US03):** Bắt buộc kích hoạt khi người dùng có nhu cầu truy xuất trạng thái dữ liệu động hoặc thống kê. Các cụm từ khóa kích hoạt thường là: *"Bao nhiêu?"*, *"Hôm nay có...?"*, *"Danh sách nào?"*.

Việc phân chia ranh giới này giúp giới hạn không gian tìm kiếm của từng luồng con, từ đó cải thiện đáng kể tốc độ truy hồi và tính chính xác của dữ liệu đầu ra.

3.2 Rào chắn định tuyến dựa trên ngữ cảnh giao diện (UI Routing Guardrail)

Một trong những thách thức lớn nhất của các hệ thống doanh nghiệp (Enterprise Systems) là hiện tượng trùng lặp nhãn giao diện (Label Collision) giữa các phân hệ khác nhau. Để tránh việc Chatbot giải thích sai ngữ cảnh, hệ thống áp dụng một rào chắn định tuyến (guardrail) vô cùng khắt khe trước khi quyết định ép (force) truy vấn vào luồng US02.

Hệ thống áp dụng nguyên lý Zero-Trust đối với dữ liệu ngữ cảnh tĩnh. Cụ thể, các tham số như `screen_id` và `module` do trình duyệt của người dùng gửi lên không được tin tưởng trực tiếp. Thay vào đó, Assistant Core thực hiện đối chiếu tham số `route` với một danh sách cho phép (allowlist) được định nghĩa trong UI Screen Registry.

Luật định tuyến chỉ kích hoạt luồng US02 khi và chỉ khi hội tụ đồng thời 5 điều kiện tiên quyết:

1. Tham số `route` được đối chiếu thành công và tồn tại trong screen registry hợp lệ.
2. Nhãn (label) được nhắc đến trong câu hỏi phải khớp chính xác với một định nghĩa UI concept (ví dụ: `approved`) trên đúng màn hình đó.
3. Ý định của câu hỏi phải thuộc nhóm giải thích khái niệm, công thức, quy tắc hoặc mối quan hệ.
4. Hoàn toàn không có tín hiệu/ý định hỏi về số liệu thực tế đang tồn tại trong cơ sở dữ liệu.
5. Hoàn toàn không có tín hiệu yêu cầu hướng dẫn thao tác quy trình.

Chỉ khi vượt qua được màng lọc logic 5 bước này, hệ thống mới giới hạn không gian ngữ cảnh theo đúng module hiện tại của người dùng. Điều này ngăn chặn triệt để rủi ro mang râu ông nọ cắm cằm bà kia (ví dụ: giải thích trạng thái "Chờ duyệt" của phân hệ Bán buôn bằng logic của phân hệ Nhập kho).

3.3 Cơ chế Thực thi song song đa luồng (Parallel Tool Execution)

Người dùng trong thực tế hiếm khi đặt những câu hỏi đơn nhiệm. Khả năng giải quyết các truy vấn phức hợp (compound queries) là một điểm sáng trong kiến trúc của Chatbot SCP. Thay vì xử lý tuần tự từng phần, bộ điều phối hỗ trợ cơ chế chọn nhiều luồng (multi-tool selection) và thực thi chúng hoàn toàn song song (parallel execution).

Ví dụ điển hình với câu hỏi phức hợp: *"Tháng này có bao nhiêu đơn chờ duyệt và Chờ duyệt nghĩa là gì?"*.

- Bộ điều phối (tác vụ `light`) sẽ bóc tách câu hỏi thành hai tiến trình độc lập.
- Tiến trình 1: Chuyển ý định *"Tháng này có bao nhiêu đơn..."* cho luồng US03 để sinh mã Text-to-SQL và đếm số lượng từ Tầng ngữ nghĩa.
- Tiến trình 2: Chuyển ý định *"...Chờ duyệt nghĩa là gì?"* cho luồng US02 để Agent truy hồi ngữ nghĩa, trích xuất cấu trúc trạng thái từ Knowledge Base.

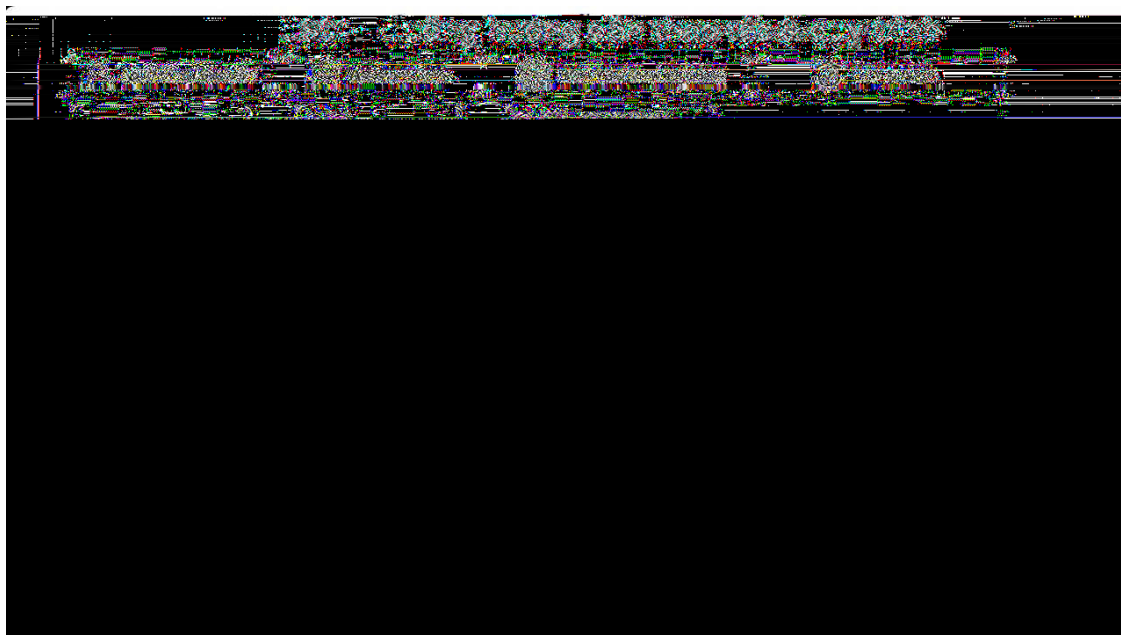
Cả hai tiến trình này diễn ra đồng thời (concurrently). Kết quả trả về từ US02 và US03 sau đó sẽ được luồng tổng hợp (synthesis) kết nối lại thành một thông điệp thống nhất và truyền tải về phía client dưới dạng Server-Sent Events (SSE).

Hơn nữa, tính độc lập của các luồng cũng được thể hiện ở cấp độ cấu hình hạ tầng. Kiến trúc quy định rõ: sự cố chưa sẵn sàng của một tool (ví dụ luồng US01 bị thiếu port kết nối) sẽ không làm tắc nghẽn hay chặn đứng hoạt động của các tool khác (như US02 và US03). Tuy nhiên, để đảm bảo chất lượng phản hồi, nếu toàn bộ các tool đều không đạt trạng thái sẵn sàng, Assistant Core sẽ chủ động từ chối khởi động (refuse to boot). Thiết kế này triệt tiêu rủi ro "bóng ma hệ thống nơi backend vẫn báo trạng thái hoạt động bình thường (health xanh) nhưng lại trả lời rỗng cho mọi truy vấn của người dùng.

Phần 4. PHÂN TÍCH LUỒNG US01 - TRỢ LÝ HƯỚNG DẪN THAO TÁC

Luồng US01 (Search Docs) đóng vai trò là một hệ chuyên gia nội bộ, chuyên trách giải quyết các truy vấn liên quan đến giao diện người dùng và quy trình vận hành. Mục tiêu thiết kế của phân hệ này không chỉ là tìm kiếm chính xác mà còn phải tối ưu hóa triệt để độ trễ (latency) và loại bỏ hoàn toàn rủi ro ảo giác (hallucination) của Mô hình ngôn ngữ.

4.1 Đặc tả luồng xử lý và Ranh giới nghiệp vụ



Hình 1: Sơ đồ khối luồng xử lý và truy hồi tài liệu thao tác (US01 Pipeline)

Phân hệ US01 (định danh công cụ: `search_docs`) được định vị là một phân hệ chuyên trách trong việc xử lý các truy vấn liên quan đến hướng dẫn thao tác trên giao diện (UI) và quy trình nghiệp vụ của người dùng[cite: 1]. Khác với các luồng phân tích logic hay truy vấn cơ sở dữ liệu, US01 vận hành theo mô hình Hệ thống hỏi đáp dựa trên tài liệu tĩnh (Document-based QA), lấy nguồn tri thức từ các cẩm nang vận hành đã được chuẩn hóa.

Điều kiện khởi tạo và Tính khả dụng (Bootstrapping Conditions): Tại thời điểm hệ thống khởi chạy (runtime), luồng US01 không được kích hoạt một cách tự động mà phải vượt qua khâu kiểm tra ràng buộc phụ thuộc. Theo thiết kế kiến trúc, công cụ này chỉ được phép kết nối (mount) vào lõi Assistant Core khi đáp ứng đầy đủ ba thành phần kỹ thuật: cổng kết nối tài liệu (docs port) đang mở, bộ mã hóa vector (BGE-M3/embedder) sẵn sàng, và kênh ghi nhật ký kiểm toán (audit sink) hoạt động ổn định[cite: 1]. Cơ chế ràng buộc này nhằm đảm bảo US01 không bao giờ phục vụ người dùng trong trạng thái thiếu khả năng lưu vết hoặc mất kết nối với cơ sở tri thức.

Ranh giới ngữ nghĩa và Cấu hình Định tuyến: Để đảm bảo tính chuyên biệt và tránh chồng chéo với các tác tử khác, hệ thống thiết lập ranh giới ngữ nghĩa cho US01 vô cùng khắt khe thông qua Bộ điều phối trung tâm. Phân hệ này chỉ được cấp phép tiếp nhận các câu hỏi mang ý định thực thi hoặc quy trình thao tác, điển hình với các cụm từ khóa như: "*Làm sao tạo...*", "*cách gửi...*", "*import như thế nào...*", hoặc "*bấm vào đâu...*"[cite: 1].

Ngược lại, nó bị cô lập hoàn toàn khỏi các truy vấn mang tính chất giải thích hệ thống (ví dụ: "*X là gì?*", "*X tính thế nào?*" thuộc thẩm quyền của luồng US02) hoặc các yêu cầu truy xuất dữ

liệu động (ví dụ: "*Bao nhiêu?*", "*Danh sách nào?*" thuộc thẩm quyền của luồng US03)[cite: 1]. Việc giới hạn ranh giới này giúp thu hẹp không gian tìm kiếm (search space), giảm thiểu nhiễu và tối ưu hóa độ chính xác cho kết quả truy hồi.

Cơ chế An toàn (Fail-safe) và Ngăn chặn Ảo giác: Trong các hệ thống quản lý chuỗi cung ứng quy mô lớn (Enterprise SCP), việc hướng dẫn sai thao tác UI có thể dẫn đến các hậu quả nghiêm trọng về sai lệch dữ liệu. Nhận thức được rủi ro này, thiết kế của US01 tuân thủ tuyệt đối nguyên tắc "từ chối suy diễn".

Trong trường hợp thuật toán truy hồi không tìm thấy bất kỳ tài liệu hướng dẫn nào khớp với độ tin cậy tối thiểu, phân hệ US01 được lập trình cứng để trả về trạng thái lỗi `not_found`[cite: 1]. Bằng rào chắn này, hệ thống tước bỏ hoàn toàn khả năng Mô hình ngôn ngữ lớn (LLM) tự ý bịa đặt (hallucinate) hay chấp vá ra một quy trình thao tác không hề tồn tại trên hệ thống SCP[cite: 1]. Đây là chốt chặn cuối cùng bảo vệ tính toàn vẹn của nghiệp vụ vận hành.

4.2 Cơ chế mã hóa ngữ nghĩa (Embedding) với BGE-M3

Để thuật toán máy tính có thể hiểu và so sánh ý định của người dùng với cơ sở tri thức tĩnh, toàn bộ truy vấn đầu vào cần được chuyển đổi thành không gian biểu diễn toán học (vector space). Phân hệ US01 sử dụng mô hình BGE-M3 làm lõi nhúng (embedding engine), đóng vai trò then chốt trong việc nhận diện các cách diễn đạt gần nghĩa[cite: 1]. Việc tích hợp mô hình này được thiết kế với những toan tính kỹ thuật nghiêm ngặt về hiệu năng và sự ổn định của toàn hệ thống.

1. Chiến lược triển khai hạ tầng (On-premise CPU Deployment):

Khác với luồng xử lý ngôn ngữ tạo sinh (synthesis) phụ thuộc vào API của nhà cung cấp bên ngoài (như OpenAI), lõi nhúng BGE-M3 được kiến trúc sư hệ thống quyết định triển khai nội bộ và vận hành hoàn toàn trên môi trường CPU[cite: 1]. Quyết định kiến trúc này giải quyết được hai bài toán cốt lõi:

- **Tối ưu hóa chi phí (Cost-efficiency):** Việc chạy BGE-M3 nội bộ giúp toàn bộ quá trình truy hồi tài liệu cho luồng US01 (và cả US02) không tiêu tốn hạn mức (quota) token của các API LLM thương mại[cite: 1]. Điều này làm giảm đáng kể chi phí vận hành (OPEX) khi hệ thống phải xử lý lượng lớn truy vấn song song.
- **Bảo mật luồng dữ liệu:** Các truy vấn chứa thông tin thao tác đặc thù của doanh nghiệp không bị gửi ra ngoài mạng internet tại bước tìm kiếm, giảm thiểu rủi ro rò rỉ dữ liệu.

2. Xử lý nhiều ngôn ngữ đầu vào (Input Pre-processing):

Đặc thù thao tác của người dùng trên các hệ thống doanh nghiệp tại Việt Nam là việc nhập liệu thường xuyên thiếu chuẩn hóa, đặc biệt là thói quen gõ tiếng Việt không dấu. Nếu chỉ mã hóa câu hỏi gốc, độ lệch vector ngữ nghĩa sẽ rất lớn.

Để khắc phục, lõi Assistant Core thực hiện một bước tiền xử lý: nó tự động tạo ra các biến thể không dấu của câu hỏi, sau đó nhúng (embed) đồng thời cả câu hỏi gốc và các biến thể này thông qua BGE-M3[cite: 1]. Chiến lược nhúng kép này giúp mở rộng không gian tìm kiếm, đảm bảo ma trận vector bao phủ được toàn bộ ý định truy vấn ngay cả khi đầu vào bị nhiễu.

3. Cơ chế chịu lỗi và Giảm cấp hệ thống (Fault Tolerance & Graceful Degradation):

Trong thiết kế hệ thống phân tán, việc một thành phần tính toán nặng như BGE-M3 hoặc cơ sở dữ liệu pgvector gặp sự cố (quá tải, nghẽn cổ chai) là rủi ro hiện hữu. Hệ thống SCP Chatbot được thiết kế với nguyên lý chịu lỗi cao (high fault tolerance).

Cụ thể, nếu mô hình BGE-M3 hoặc kênh vector gặp lỗi sinh vector, hệ thống không đổ vỡ hoàn toàn (crash) mà sẽ tự động chuyển sang trạng thái giảm cấp. Runtime của Assistant Core sẽ khai báo cờ cảnh báo `degraded_channels=["vector"]`[cite: 1]. Dưới trạng thái này, luồng US01 vẫn tiếp tục duy trì hoạt động truy hồi thông qua hai kênh dự phòng là Tìm kiếm chính xác (Exact match) và Tìm kiếm từ khóa thừa (BM25)[cite: 1]. Cơ chế "Graceful Degradation" này đảm bảo tính khả dụng (availability) của hệ thống luôn ở mức tối đa, bảo vệ trải nghiệm tra cứu liên tục của người dùng.

4.3 Thuật toán tìm kiếm lai (Hybrid Retrieval) và Lọc ứng viên

Trong các hệ thống quản trị doanh nghiệp, tài liệu hướng dẫn thường chứa cả các thuật ngữ kỹ thuật cứng (như mã màn hình, tên nút bấm) lẫn các đoạn mô tả quy trình bằng ngôn ngữ tự nhiên. Việc chỉ sử dụng một phương pháp tìm kiếm đơn lẻ sẽ bộc lộ nhiều điểm yếu: tìm kiếm từ khóa (Keyword-based) dễ bỏ sót các từ đồng nghĩa, trong khi tìm kiếm ngữ nghĩa (Semantic-based) lại thường tính toán sai độ lệch đối với các định danh chính xác. Để giải quyết triệt để bài toán này, luồng US01 triển khai thuật toán tìm kiếm lai (Hybrid Retrieval).

1. Truy hồi đa chiều (Multi-dimensional Retrieval):

Thuật toán tìm kiếm lai của hệ thống vận hành bằng cách quét không gian cơ sở tri thức trên hai trục độc lập nhưng bổ trợ cho nhau:

- **Trục Vector thưa (Sparse Vector):** Đại diện cho thuật toán khớp từ khóa (thường dựa trên BM25), trục này chịu trách nhiệm tìm kiếm các tài liệu chứa chính xác các thuật ngữ, tên nhãn giao diện hoặc cụm từ khóa có trong truy vấn của người dùng[cite: 1].
- **Trục Vector mật độ cao (Dense Vector):** Sử dụng ma trận nhúng từ BGE-M3, trục này đo lường khoảng cách tương đồng ngữ nghĩa (Cosine Similarity hoặc L2 Distance) giữa câu hỏi và tài liệu, giúp hệ thống nhận diện được các kết quả phù hợp ngay cả khi người dùng sử dụng từ ngữ diễn đạt khác với tài liệu gốc[cite: 1].

Quá trình truy hồi song song này thu thập tất cả các đoạn tài liệu (chunks) tiềm năng thành một tập

ứng viên (candidate pool) ban đầu.

2. Thuật toán Lọc cắt tỉa (Candidate Filtering/Pruning):

Tập ứng viên thô thu được từ bước truy hồi lại thường chứa lượng lớn thông tin nhiễu (noise) do sự kết hợp của nhiều kênh tìm kiếm. Nếu đẩy toàn bộ tập dữ liệu này vào cửa sổ ngữ cảnh (context window) của LLM, hệ thống sẽ đối mặt với rủi ro pha loãng ngữ cảnh (context dilution) và lãng phí tài nguyên tính toán.

Vì vậy, Assistant Core thiết lập một bộ lọc cắt tỉa (Pruning filter) hoạt động như một rào chắn chất lượng. Hệ thống tiến hành đánh giá lại điểm số của từng ứng viên và tự động lọc bỏ các tài liệu được xác định là quá xa về mặt ngữ nghĩa so với truy vấn gốc[cite: 1]. Thuật toán này áp dụng một ngưỡng (threshold) điểm tnh hoặc động; mọi ứng viên có độ tin cậy nằm dưới ngưỡng này đều bị loại trừ ngay lập tức.

3. Tác động của bộ lọc đến tính ổn định hệ thống:

Cơ chế lọc ứng viên không chỉ có ý nghĩa về mặt hiệu năng mà còn đóng vai trò cốt lõi trong việc đảm bảo an toàn thông tin. Nếu bộ lọc loại bỏ toàn bộ ứng viên trong tập thô (tức là không có tài liệu nào vượt qua được ngưỡng tin cậy), luồng US01 sẽ ngay lập tức ngắt quá trình xử lý và trả về trạng thái `not_found`[cite: 1]. Rào chắn này đảm bảo dữ liệu cung cấp cho người dùng hoặc LLM luôn là dữ liệu có căn cứ vững chắc, chặn đứng hoàn toàn khả năng hệ thống trả về các hướng dẫn không liên quan hoặc tự bịa đặt quy trình.

4.4 Chiến lược kết xuất trực tiếp (Direct Render Bypass) tối ưu độ trễ

Trong các kiến trúc RAG (Retrieval-Augmented Generation) truyền thống, pha tạo sinh (synthesis) do LLM đảm nhiệm thường là nút thắt cổ chai (bottleneck) lớn nhất của toàn bộ hệ thống. Quá trình gửi ngữ cảnh qua mạng và chờ LLM sinh từng token (Time to First Token - TTFT) tạo ra độ trễ (latency) tính bằng giây, đồng thời tiêu tốn một lượng lớn tài nguyên tính toán (quota). Đứng từ góc độ tối ưu hóa hệ thống, việc sử dụng một mạng nơ-ron khổng lồ chỉ để diễn đạt lại một đoạn tài liệu hướng dẫn vốn đã được viết rất chuẩn mực là một sự lãng phí nghiêm trọng về mặt kiến trúc.

Để giải quyết triệt để điểm nghẽn này, phân hệ US01 được thiết kế tích hợp một cơ chế đoản mạch (short-circuit) mang tên Kết xuất trực tiếp (Direct Render Bypass). Cơ chế này hoạt động dựa trên việc phân tích phân phối điểm số tin cậy (confidence score distribution) của tập tài liệu ứng viên sau quá trình truy hồi lại.

Logic kích hoạt Bypass:

Cơ chế kiểm soát luồng của lõi Assistant Core sẽ đánh giá khoảng cách điểm số giữa ứng viên top đầu và các ứng viên còn lại. Nếu hệ thống nhận định có một hướng dẫn đạt mức điểm trội rõ ràng (absolute confidence) so với phần còn lại, Core sẽ tiến hành render kết quả này trực tiếp ra giao

diện người dùng[cite: 1]. Trong kịch bản này, hệ thống sẽ bỏ qua hoàn toàn bước gọi LLM[cite: 1].

Điều kiện chuyển đổi (Context Switching):

Thuật toán Bypass được thiết kế với tính chọn lọc rất cao. Nó chỉ được kích hoạt khi hệ thống đánh giá rằng một tài liệu duy nhất đã đủ để giải quyết trọn vẹn truy vấn. Ngược lại, nếu câu hỏi của người dùng phức tạp và đòi hỏi phải ghép nối quy trình từ nhiều bước khác nhau, cơ chế bypass sẽ tự động bị vô hiệu hóa[cite: 1]. Khi đó, hệ thống trở về luồng tiêu chuẩn: chỉ các đoạn tài liệu đã được chọn lọc mới được đóng gói và gửi sang LLM để thực hiện tổng hợp văn bản[cite: 1].

Đánh giá tác động đến hiệu năng hệ thống:

Việc ứng dụng chiến lược Direct Render Bypass mang lại ba giá trị cốt lõi cho hệ thống phân tán của SCP:

- **Triệt tiêu độ trễ tạo sinh:** Thời gian phản hồi (Response Time) đối với các câu hỏi FAQ thao tác cơ bản giảm từ mức tính bằng giây (của LLM) xuống mức mili-giây (thời gian truy vấn cơ sở dữ liệu vector).
- **Bảo toàn Quota Token:** Bằng cách không gọi API LLM cho các tác vụ không cần thiết, hệ thống tiết kiệm được đáng kể chi phí vận hành (OPEX) tại các thời điểm tải hệ thống đạt đỉnh (peak time).
- **Đạt tính định định tuyệt đối (Strict Determinism):** Do kết quả được trích xuất nguyên bản 100% từ cơ sở tri thức tĩnh đã được phê duyệt, tỷ lệ rủi ro ảo giác (hallucination) đối với các luồng thao tác được kích hoạt bypass giảm xuống chính xác bằng 0.

4.5 Cơ chế tổng hợp đa bước và Ràng buộc kiểm tra (Grounding Check)

Trong thực tế vận hành hệ thống SCP, nhiều truy vấn của người dùng không thể được giải quyết bằng một tài liệu đơn lẻ mà đòi hỏi sự kết hợp của nhiều thao tác phân mảnh. Khi kịch bản này xảy ra, cơ chế kết xuất trực tiếp (Direct Render) bị vô hiệu hóa, và hệ thống chuyển trạng thái sang luồng tổng hợp đa bước (Multi-step Synthesis). Thách thức lớn nhất của luồng này là làm sao để LLM xâu chuỗi thông tin một cách tự nhiên nhưng tuyệt đối không được tự ý "sáng tác" thêm các thao tác không có trong cẩm nang.

Để khống chế rủi ro này, luồng US01 triển khai một kiến trúc kiểm định hai lớp (Two-layer Validation Architecture):

1. Đóng gói Ngữ cảnh Tĩnh (Strict Context Bounding):

Thay vì cung cấp cho LLM toàn bộ tri thức hoặc để nó tự do truy xuất, hệ thống áp dụng nguyên tắc đóng gói ngữ cảnh nghiêm ngặt. Khi cần ghép nối nhiều bước hướng dẫn, lõi Assistant Core sẽ đóng vai trò người gác cổng: chỉ những đoạn tài liệu (chunks) đã vượt qua vòng lọc ứng viên mới

được phép đóng gói và gửi sang LLM[cite: 1]. Giới hạn "tầm nhìn" vật lý này ép buộc LLM phải hoạt động trong một không gian thông tin hữu hạn, cắt đứt khả năng nội suy lan man của mô hình.

2. Truy vết Nguồn gốc (Traceability) trong quá trình Tổng hợp:

LLM được cấp một prompt hệ thống (system prompt) chuyên biệt cho tác vụ *synthesis*. Nó có nhiệm vụ làm mượt ngôn từ và liên kết các đoạn tài liệu rời rạc thành một quy trình *step-by-step* liền mạch. Tuy nhiên, một ràng buộc cứng được thiết lập: LLM bắt buộc phải chỉ ra rõ ràng các bước mà nó đã sử dụng (thông qua cơ chế đánh dấu ID tài liệu)[cite: 1]. Điều này đảm bảo mọi luận điểm do LLM phát sinh đều có dấu vết truy xuất (*provenance tracking*) rõ ràng.

3. Rào chắn Đối chiếu Độc lập (Independent Grounding Check):

Đây là chốt chặn an ninh cuối cùng trước khi phản hồi được truyền tải (*stream*) về phía client. Thay vì tin tưởng hoàn toàn vào kết quả sinh ra của LLM, Assistant Core chạy một tiến trình kiểm tra đối chiếu (*Grounding Check*) độc lập[cite: 1]. Tiến trình này sẽ bóc tách các trích dẫn (*citation*) mà LLM đính kèm và đối chiếu ngược lại với danh sách các đoạn tài liệu gốc (*chunks*) đã cung cấp ban đầu[cite: 1].

Đánh giá tác động Hệ thống:

Nếu hàm kiểm tra (*Grounding Check*) phát hiện LLM chêm thêm một thao tác lạ, hoặc vi phạm tính toàn vẹn của dữ liệu gốc, lõi hệ thống sẽ bác bỏ hoàn toàn kết quả này. Thiết kế này tuân thủ triệt để nguyên tắc: nếu không tìm thấy tài liệu căn cứ vững chắc, phân hệ US01 sẽ chủ động trả về trạng thái *not_found* thay vì để hệ thống tự bịa ra một quy trình không tồn tại[cite: 1]. Nhờ kiến trúc này, rủi ro ảo giác (*hallucination*) trong luồng hướng dẫn thao tác được kiểm soát ở cấp độ hệ thống (*system-level*) chứ không chỉ phụ thuộc vào năng lực của bản thân LLM.

Phần 5. PHÂN TÍCH LUỒNG US 2 - Giải thích Hệ thống

5.1 Mục đích và phạm vi US02

US02 là luồng xử lý câu hỏi thuộc nhóm giải thích ý nghĩa hệ thống trong Chatbot SCP. Luồng này chịu trách nhiệm diễn giải cho người dùng những gì họ nhìn thấy trên giao diện và logic nghiệp vụ tương ứng phía sau, bao gồm: trạng thái, nhãn dữ liệu, công thức tính toán, quy tắc nghiệp vụ (*rule*), quan hệ dữ liệu (*relation*) và ánh xạ API khi cần thiết.

Khác với việc trả lời tự do dựa trên tri thức nền của mô hình ngôn ngữ lớn (LLM), US02 được thiết kế theo nguyên tắc *grounded generation*: mọi câu trả lời phải được tổng hợp từ một tập *evidence* đã được truy hồi và phê duyệt trước, không cho phép LLM tự suy diễn ngoài phạm vi *corpus*. *Corpus* phục vụ US02 gồm năm nhóm dữ liệu: *Evidence backend*, *Formula*, *Rule*, *Relation* và *UI semantic*.

Phạm vi của US02 giới hạn ở việc *giải thích*, không bao gồm:

- Thực hiện hoặc hướng dẫn các bước thao tác trên giao diện (thuộc US01);
- Truy vấn và trả về số liệu thực tế đang tồn tại trong hệ thống (thuộc US03).

Việc phân định phạm vi rõ ràng này là điều kiện tiên quyết để hệ thống định tuyến (routing) hoạt động chính xác, tránh tình trạng một câu hỏi bị xử lý sai luồng dẫn đến câu trả lời không phù hợp với nhu cầu người dùng.

5.2 Ranh giới với US01 và US03

Ba luồng US01, US02 và US03 được phân định dựa trên *bản chất câu hỏi* chứ không dựa trên *chủ đề* hay *module nghiệp vụ*. Điều này có nghĩa là cùng một chủ đề nghiệp vụ có thể được xử lý bởi ba luồng khác nhau tùy theo cách người dùng đặt câu hỏi, được minh họa qua Bảng 1.

Bảng 1: Ranh giới định tuyến giữa US01, US02, US03 theo dạng câu hỏi

Dạng câu hỏi	Luồng	Ví dụ
Giải thích ý nghĩa, công thức, quan hệ	US02	“Giá bán buôn được tính thế nào?”
Hướng dẫn thao tác	US01	“Làm sao tạo giá bán buôn?”
Tra cứu số liệu thực tế	US03	“Giá bán buôn của đơn này là bao nhiêu?”

Như minh họa ở Bảng 1, cùng chủ đề “giá bán buôn” nhưng ba cách đặt câu hỏi khác nhau dẫn đến ba luồng xử lý hoàn toàn khác nhau về nguồn dữ liệu và cơ chế xử lý:

- US02 truy hồi evidence từ corpus (Formula, Rule) và tổng hợp câu trả lời có trích dẫn, không truy cập dữ liệu giao dịch thực tế.
- US01 truy hồi tài liệu hướng dẫn thao tác từ Knowledge Base, không diễn giải logic nghiệp vụ.
- US03 sinh truy vấn SQL trên semantic view đã được phê duyệt để trả về số liệu cụ thể.

Để đảm bảo việc định tuyến chính xác giữa ba luồng, hệ thống áp dụng một tầng *tool selector* bằng LLM ở mức tác vụ `Light` nhằm phân loại ý định ban đầu, sau đó bổ sung một rào chắn xác định bằng luật (rule-based guardrail) riêng cho US02 nhằm hạn chế rủi ro định tuyến sai khi nhân UI trùng lặp giữa nhiều màn hình. Cơ chế phân tách rạch ròi này là nền tảng để đảm bảo mỗi câu hỏi được xử lý bởi đúng nguồn dữ liệu và đúng cơ chế kiểm soát chất lượng tương ứng.

5.3 Kiến trúc hệ thống

5.3.1 Vị trí US02 trong tổng thể Assistant Core

US02 là một trong ba luồng xử lý được cắm (plug-in) vào Assistant Core — thành phần trung tâm điều phối chatbot, vận hành độc lập ở cổng 8000 và tách biệt hoàn toàn khỏi SCP Frontend (3001) và SCP Backend (3002). Assistant Core được thiết kế như một API sản phẩm dùng chung: bất kỳ giao diện nào (SCP, web gateway, hoặc client khác) đều có thể gọi vào Core thông qua cùng một hợp đồng API, không có thiết kế ràng buộc riêng cho một frontend cụ thể.

Trong luồng xử lý tổng thể, US02 nằm sau tầng *tool selector* — nơi một LLM ở mức tác vụ Light xác định câu hỏi thuộc luồng nào (US01, US02, US03) và có thể kích hoạt song song nhiều luồng nếu câu hỏi mang nhiều ý định. Sau khi được định tuyến vào US02, luồng này hoạt động như một *Agent* riêng biệt, có planner và cơ chế truy hồi (retrieval) chuyên biệt, khác với US01 (truy hồi tài liệu hướng dẫn) và US03 (sinh và thực thi SQL).

Về mặt phụ thuộc vận hành, US02 chỉ được kích hoạt khi hai điều kiện được thỏa mãn khi Core khởi động: có *evidence catalog* hợp lệ và có *audit sink* sẵn sàng. Nếu điều kiện này chưa đủ, US02 không chặn các luồng khác hoạt động — nguyên tắc mà Core áp dụng chung cho cả ba luồng nhằm tránh một luồng lỗi kéo sập toàn bộ hệ thống.

5.3.2 Corpus runtime — 5 nhóm dữ liệu

Khác với US01 (dựa trên tài liệu hướng dẫn thao tác) và US03 (dựa trên semantic view của database), US02 vận hành trên một *corpus* riêng gồm năm nhóm dữ liệu đã được trích xuất và phê duyệt từ nguồn hệ thống, được tóm tắt trong Bảng 2.

Bảng 2: Năm nhóm dữ liệu trong corpus runtime của US02

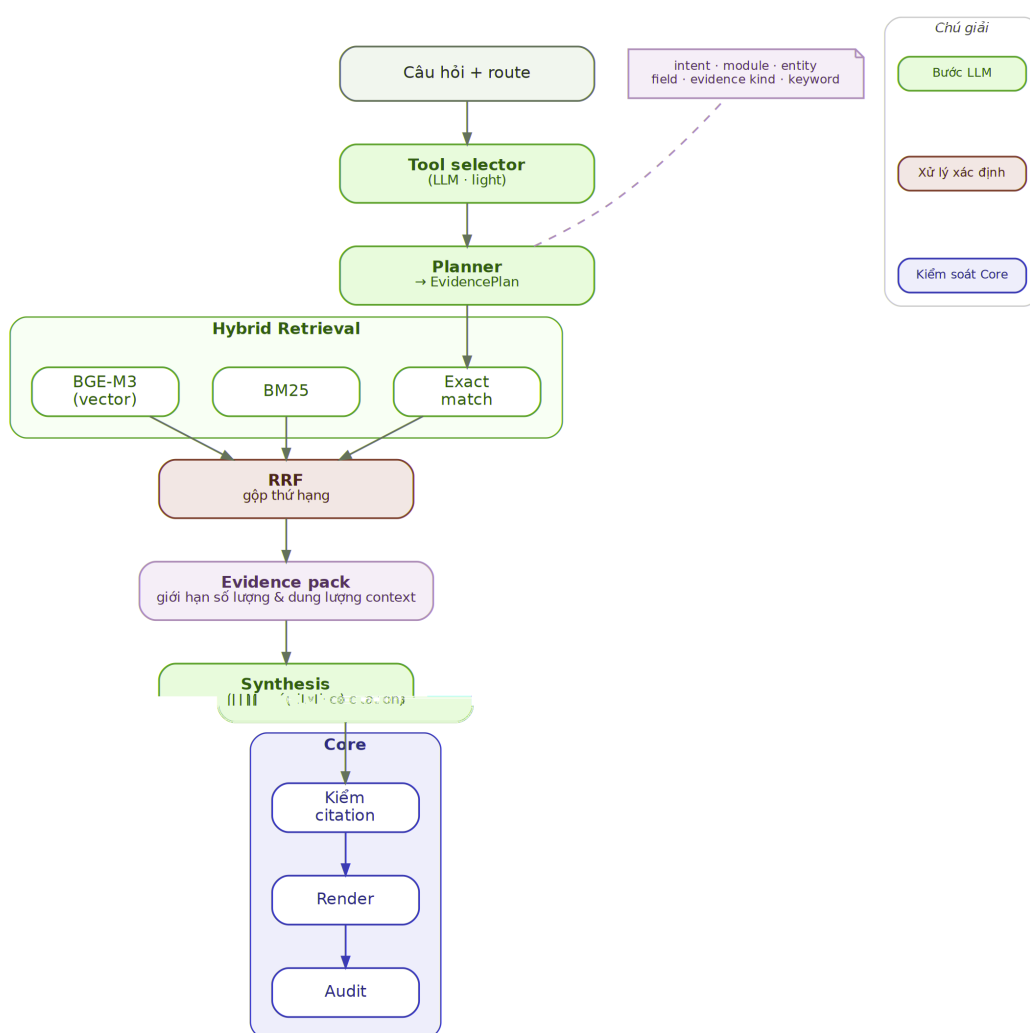
Nhóm dữ liệu	Mô tả
Evidence backend	API, field, validation, business rule và test case được trích xuất trực tiếp từ mã nguồn (source)
Formula	Các biểu thức nghiệp vụ đã được phê duyệt (công thức tính toán)
Rule	Các điều kiện và chính sách nghiệp vụ
Relation	Quan hệ giữa dữ liệu, entity và module
UI semantic	Ý nghĩa của nhãn, nút bấm, trạng thái, filter và chỉ số hiển thị trên giao diện

Cần lưu ý một điểm quan trọng trong thiết kế: các *exclusion UI* — tức các nhãn giao diện đã được xác nhận là không cần giải thích — chỉ phục vụ mục đích kiểm soát độ phủ (coverage), không phải là nguồn dữ liệu để LLM sử dụng khi tổng hợp câu trả lời. Việc tách bạch này giúp đảm bảo

mọi câu trả lời của US02 luôn xuất phát từ nội dung đã được duyệt làm evidence, thay vì suy diễn từ danh sách loại trừ.

5.3.3 Sơ đồ luồng xử lý (Agent flow)

Một lượt xử lý hoàn chỉnh của US02 trải qua năm giai đoạn chính.



Hình 2: Sơ đồ luồng xử lý (Agent flow) của US02

Mỗi giai đoạn trong sơ đồ đảm nhận một vai trò kiểm soát riêng:

- Tool selector chỉ chọn tool và trích entity/module ban đầu, không trả lời trực tiếp người dùng.
- Planner chỉ chọn intent, module, entity, field, loại evidence và từ khóa cần truy hồi (đóng gói thành EvidencePlan) — planner *không* tự tổng hợp câu trả lời.

-
- **Hybrid Retrieval** kết hợp ba kênh Exact, BM25 và BGE-M3 để vừa khớp chính xác nhãn/field/API, vừa bắt được các biến thể ngôn ngữ gần nghĩa.
 - **RRF** (Reciprocal Rank Fusion) gộp thứ hạng kết quả từ ba kênh thành một danh sách duy nhất.
 - **Evidence pack** là tập hợp các evidence đã qua giới hạn về số lượng và dung lượng, được gửi cho LLM ở bước synthesis — LLM *chỉ* được nhìn thấy nội dung trong evidence pack này, không có quyền truy cập corpus đầy đủ.
 - **Synthesis** tổng hợp câu trả lời, mỗi luận điểm (claim) phải tham chiếu đến một hoặc nhiều evidence ID cụ thể trong pack.
 - **Core** thực hiện bước kiểm tra cuối: từ chối mọi citation nằm ngoài evidence pack trước khi render câu trả lời và ghi audit.

Thiết kế theo từng giai đoạn tách biệt như trên cho phép kiểm soát được nguồn gốc của mọi thông tin xuất hiện trong câu trả lời cuối cùng, đồng thời tạo điều kiện để audit và truy vết lỗi ở từng bước riêng lẻ khi cần điều tra sự cố.

5.4 Cơ chế định tuyến vào US02

Trước khi một câu hỏi được xử lý bởi US02, hệ thống phải xác định chính xác rằng câu hỏi đó thực sự thuộc phạm vi giải thích ý nghĩa/logic hệ thống, chứ không phải thuộc US01 (thao tác) hay US03 (số liệu). Việc định tuyến sai không chỉ khiến câu trả lời không đúng nhu cầu người dùng, mà còn có nguy cơ khiến LLM trả lời dựa trên nhãn UI sai ngữ cảnh — đặc biệt khi cùng một nhãn xuất hiện ở nhiều màn hình với ý nghĩa khác nhau. Vì vậy, việc định tuyến vào US02 được thực hiện qua hai lớp liên tiếp: một lớp phân loại bằng LLM và một lớp rào chắn xác định bằng luật.

5.4.1 Vai trò *tool selector* (LLM light)

Lớp đầu tiên là *tool selector*, sử dụng LLM ở mức tác vụ **Light**. Vai trò của thành phần này giới hạn ở hai việc:

- Chọn (các) tool phù hợp với câu hỏi trong số US01, US02, US03;
- Trích xuất entity/module ban đầu từ câu hỏi để phục vụ các bước xử lý tiếp theo.

Tool selector không trả lời trực tiếp người dùng — đây là nguyên tắc thiết kế nhất quán trong toàn hệ thống, nhằm tách bạch giữa bước *định tuyến ý định* (routing) và bước *tổng hợp nội dung*

(synthesis), tránh việc một LLM duy nhất vừa quyết định luồng xử lý vừa tự trả lời không qua kiểm soát evidence.

Do sử dụng LLM để phân loại, tool selector mang bản chất xác suất chứ không tuyệt đối chính xác — đặc biệt với các câu hỏi mang nhiều ý định trong cùng một câu (ví dụ vừa hỏi số liệu vừa hỏi ý nghĩa), khi đó nhiều tool có thể được chọn và chạy song song. Tuy nhiên, đối với riêng US02, việc chỉ dựa vào một lớp phân loại bằng LLM là chưa đủ an toàn, vì nhãn UI có thể trùng tên giữa các module khác nhau. Đây là lý do hệ thống bổ sung thêm một lớp rào chắn xác định bằng luật ngay sau tool selector.

5.4.2 Rào chắn 5 điều kiện

Sau tool selector, một rào chắn xác định bằng luật (*rule-based guardrail*) được áp dụng riêng cho US02. Rào chắn này chỉ cho phép ép định tuyến sang US02 khi đồng thời thoả mãn cả năm điều kiện sau:

1. Route của câu hỏi được đối chiếu thành công với *screen registry* (danh mục màn hình hợp lệ đã đăng ký);
2. Nhãn được hỏi khớp với một UI concept ở trạng thái approved trên *đúng* màn hình tương ứng với route đó;
3. Câu hỏi thuộc nhóm giải thích/công thức/rule/relation (không phải hỏi thao tác hay số liệu);
4. Không có tín hiệu cho thấy người dùng đang hỏi số liệu thực tế;
5. Không có tín hiệu cho thấy người dùng đang hỏi hướng dẫn thao tác.

Việc yêu cầu thoả mãn *đồng thời* cả năm điều kiện — thay vì chỉ dựa vào kết quả phân loại của LLM — giúp giảm đáng kể rủi ro trả lời sai ngữ cảnh. Nếu bất kỳ điều kiện nào trong năm điều kiện trên không được thoả mãn, hệ thống không ép định tuyến vào US02, tránh tình trạng LLM "gán ghép" một nhãn hỏi vào một UI concept không đúng màn hình chỉ vì tên gọi giống nhau.

5.4.3 Vì sao không tin *screen_id/module* từ browser

Một điểm thiết kế quan trọng của rào chắn là: giá trị *screen_id* và *module* do trình duyệt (browser) gửi lên trong *page_context* của request không được tin dùng trực tiếp để xác định ngữ cảnh UI. Thay vào đó, *route* sau khi đã được đối chiếu qua allowlist của *screen registry* mới là nguồn ngữ cảnh đáng tin cậy (điều kiện thứ nhất trong Mục 3.2).

Có hai lý do chính cho quyết định thiết kế này:

- **Tránh trộn lẫn ngữ nghĩa giữa các module.** Nhiều nhãn UI có tên gọi giống hệt nhau nhưng mang ý nghĩa khác nhau tùy theo module (ví dụ: nhãn “Trạng thái” hay “Giá” có thể xuất hiện ở nhiều màn hình với ngữ nghĩa khác nhau). Nếu tin trực tiếp giá trị `screen_id/module` từ client mà không xác thực qua registry, hệ thống có nguy cơ tra cứu evidence sai module, dẫn đến câu trả lời đúng chữ nhưng sai ngữ cảnh.
- **Giảm rủi ro dữ liệu đầu vào không đáng tin cậy.** Giá trị gửi từ browser nằm ngoài tầm kiểm soát trực tiếp của Core — có thể sai lệch do lỗi client, dữ liệu cũ chưa cập nhật, hoặc bị chỉnh sửa. Việc bắt buộc đối chiếu qua screen registry (một danh mục đã được kiểm duyệt phía server) đảm bảo ngữ cảnh dùng để định tuyến luôn nằm trong tập giá trị hợp lệ đã biết trước, thay vì chấp nhận bất kỳ giá trị nào được gửi lên.

Nhờ cơ chế xác thực hai chiều này — vừa yêu cầu LLM phân loại đúng ý định câu hỏi, vừa yêu cầu ngữ cảnh UI phải được xác thực qua allowlist phía server — US02 giảm thiểu được rủi ro trả lời nhầm lẫn giữa các module khác nhau, một trong những lỗi khó phát hiện nhất đối với hệ thống hỏi-đáp dựa trên ngữ cảnh giao diện.

5.5 Cơ chế Retrieval

Sau khi Planner tạo ra EvidencePlan, hệ thống thực hiện bước truy hồi (retrieval) để tìm các evidence liên quan trong corpus trước khi đưa vào synthesis. Thay vì chỉ dựa vào một phương pháp tìm kiếm duy nhất, US02 áp dụng *hybrid retrieval* — kết hợp ba kênh tìm kiếm có bản chất khác nhau nhằm bù trừ điểm yếu cho nhau, sau đó gộp kết quả bằng thuật toán Reciprocal Rank Fusion (RRF).

5.5.1 Ba kênh: Exact, BM25, BGE-M3

Ba kênh truy hồi hoạt động song song trên cùng một truy vấn, mỗi kênh đảm nhận một vai trò riêng biệt, được tóm tắt trong Bảng 3.

Bảng 3: Vai trò của ba kênh trong Hybrid Retrieval

Kênh	Vai trò
Exact	Khớp chính xác nhãn, alias, tên field, API và identifier kỹ thuật
BM25	Khớp theo từ khoá và các biến thể từ ngữ (dựa trên tần suất xuất hiện từ)
BGE-M3	Nhận biết các diễn đạt gần nghĩa (semantic similarity) thông qua vector embedding

Ba kênh này bù trừ điểm yếu cho nhau: Exact đảm bảo độ chính xác tuyệt đối khi người dùng gõ đúng tên field/nhãn, nhưng thất bại nếu người dùng diễn đạt khác đi; BM25 xử lý được các biến

thể từ vựng nhưng vẫn phụ thuộc vào từ khoá xuất hiện trong văn bản; BGE-M3 giải quyết trường hợp người dùng hỏi bằng cách diễn đạt hoàn toàn khác nhưng cùng ý nghĩa, đánh đổi bằng việc có thể trả về kết quả kém liên quan hơn nếu không được kiểm soát ngưỡng phù hợp (xem Mục 4.3).

5.5.2 Công thức và tham số RRF

Kết quả từ ba kênh được gộp lại bằng thuật toán Reciprocal Rank Fusion (RRF), tính điểm tổng hợp cho mỗi evidence d theo công thức:

$$Score(d) = \sum_c \frac{1}{60 + rank_c(d)} \quad (2)$$

trong đó c là chỉ số của từng kênh truy hồi (Exact, BM25, BGE-M3), $rank_c(d)$ là thứ hạng của evidence d trong kết quả trả về của kênh c , và hằng số 60 (ký hiệu `rrf_k`) đóng vai trò làm giảm ảnh hưởng của các kênh có thứ hạng thấp, giúp thuật toán ổn định hơn khi một kênh trả về kết quả nhiều ở vị trí cao.

Một điểm khác biệt quan trọng so với RRF nguyên bản là: evidence đạt *exact match* được cộng thêm một điểm ưu tiên cố định ngoài công thức (2). Nhờ đó, evidence khớp chính xác nhãn/trường luôn được xếp trước evidence chỉ khớp gần nghĩa, ngay cả khi thứ hạng RRF thuần túy không phản ánh đủ mức độ ưu tiên này — một cơ chế cần thiết vì trong ngữ cảnh giải thích hệ thống, độ chính xác của thuật ngữ thường quan trọng hơn độ đa dạng ngữ nghĩa.

5.5.3 Bảng cấu hình hiện tại

Các tham số vận hành của tầng retrieval được cấu hình tập trung, cho phép điều chỉnh mà không cần thay đổi mã nguồn. Cấu hình hiện tại của US02 được tổng hợp trong Bảng 4.

Bảng 4: Cấu hình hiện tại của tầng Retrieval — US02

Tham số	Giá trị
Số evidence tối đa mỗi chunk	8
Số ký tự tối đa mỗi evidence	2.500
Tổng dung lượng evidence pack tối đa	18.000 ký tự
Kích thước vector candidate pool	40
Ngưỡng vector tối thiểu (BGE-M3)	0,45
Hằng số RRF (<code>rrf_k</code>)	60

Các giới hạn về số lượng và dung lượng evidence (dòng 1–3) đóng vai trò kiểm soát kích thước context gửi vào LLM ở bước synthesis, vừa tránh vượt giới hạn ngữ cảnh của mô hình, vừa hạn chế nhiều khi có quá nhiều evidence không thực sự liên quan. Ngưỡng vector tối thiểu 0,45 đóng

vai trò lọc bỏ các ứng viên có độ tương đồng ngữ nghĩa quá thấp trước khi đưa vào bước gộp RRF, giảm nguy cơ kênh BGE-M3 đưa vào các evidence không liên quan chỉ vì tình cờ có điểm vector cao trong candidate pool.

5.5.4 Cơ chế fallback khi vector lỗi (*degraded_channels*)

Vì kênh BGE-M3 phụ thuộc vào một thành phần embedding chạy riêng và cơ sở dữ liệu vector (pgvector), đây là kênh có nguy cơ gián đoạn cao hơn hai kênh còn lại (Exact và BM25 vốn không phụ thuộc vào mô hình embedding hay hạ tầng vector). Để tránh việc một sự cố ở tầng vector làm toàn bộ US02 ngừng hoạt động, hệ thống áp dụng cơ chế suy giảm có kiểm soát (*graceful degradation*):

- Khi BGE-M3 hoặc pgvector gặp lỗi, hai kênh Exact và BM25 vẫn tiếp tục hoạt động bình thường, đảm bảo US02 không bị gián đoạn hoàn toàn.
- Runtime khai báo tường minh cờ trạng thái `degraded_channels=["vector"]`, cho phép hệ thống (và audit log) ghi nhận rằng kết quả truy hồi ở lượt xử lý đó đã thiếu đi khả năng nhận diện diễn đạt gần nghĩa.

Cơ chế này phản ánh nguyên tắc thiết kế chung của toàn hệ thống: một thành phần con gặp sự cố không được phép kéo sập toàn bộ luồng xử lý, miễn là hệ thống vẫn còn ít nhất một kênh hoạt động đáng tin cậy. Tuy nhiên, cần lưu ý rằng ở trạng thái suy giảm, khả năng US02 tìm đúng evidence cho các câu hỏi diễn đạt khác biệt so với thuật ngữ gốc trong corpus sẽ giảm đi, do chỉ còn dựa vào khớp từ khóa (Exact, BM25) — đây là điểm cần được giám sát qua tỷ lệ `not_found` khi `degraded_channels` được kích hoạt.

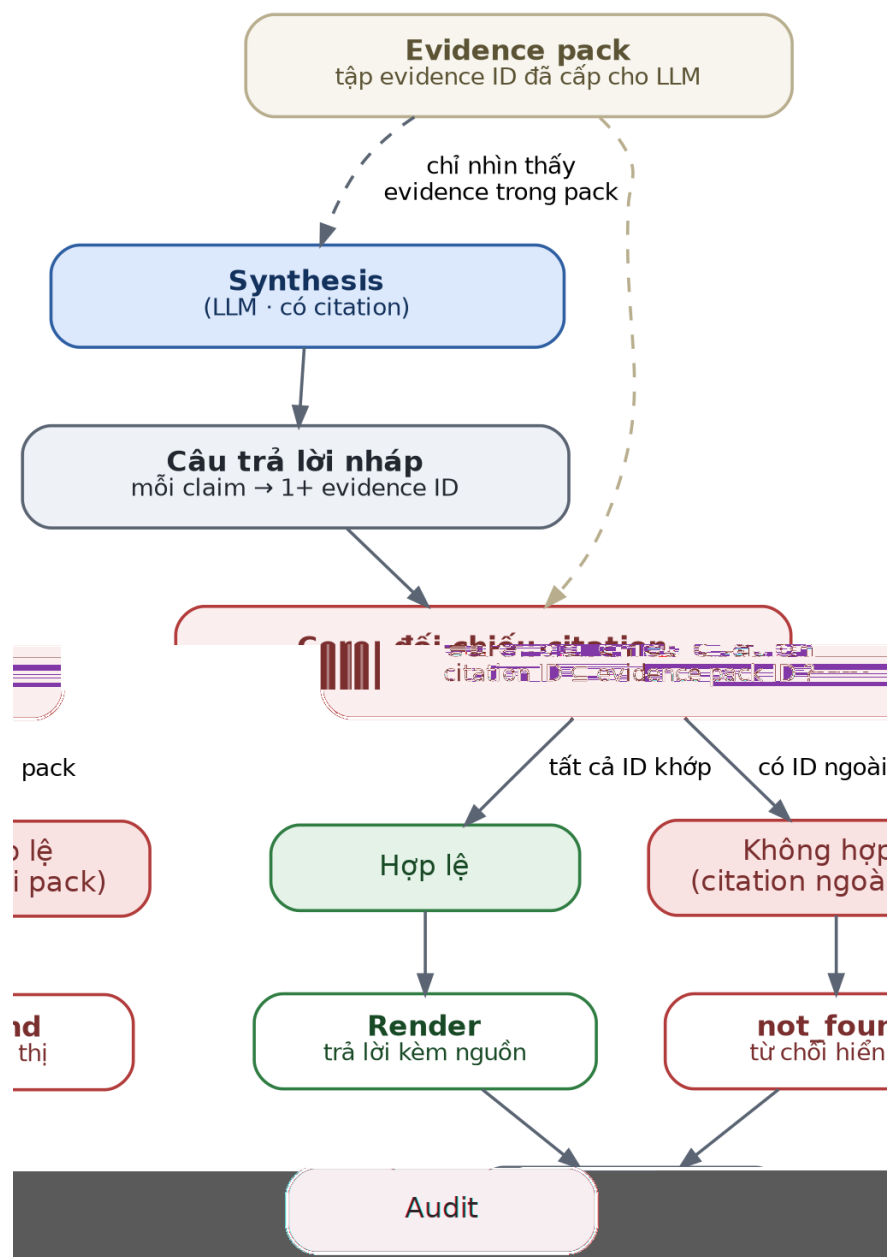
5.6 Cơ chế kiểm citation

Nguyên tắc cốt lõi của bước synthesis là: LLM chỉ được phép nhìn thấy nội dung nằm trong evidence pack đã được truy hồi và giới hạn ở Mục ??, không có quyền truy cập trực tiếp vào toàn bộ corpus hay tri thức nền của mô hình để trả lời câu hỏi. Trong quá trình sinh câu trả lời, mỗi luận điểm (*claim*) mà LLM đưa ra phải tham chiếu đến một hoặc nhiều evidence ID cụ thể có mặt trong evidence pack đó.

Sau khi LLM sinh ra câu trả lời có kèm citation, Core thực hiện một bước kiểm tra độc lập: đối chiếu từng citation được LLM trích dẫn với danh sách evidence ID thực sự có trong pack đã gửi cho LLM ở lượt xử lý đó. Đây là một bước kiểm soát tách biệt khỏi LLM, không phụ thuộc vào việc LLM có "tự giác" tuân thủ chỉ dẫn trong prompt hay không.

Core từ chối mọi citation nằm ngoài evidence pack. Nếu LLM trích dẫn một evidence ID không tồn tại trong pack đã cấp — dù đó là ID hợp lệ ở nơi khác trong corpus, hay là ID không có thật do mô hình sinh nhầm — citation đó bị coi là không hợp lệ.

Cơ chế kiểm soát này giải quyết trực tiếp một rủi ro cố hữu của các hệ thống dựa trên LLM: hiện tượng *hallucination*, khi mô hình có thể tự tin đưa ra thông tin không có thật hoặc gán nhầm nguồn cho một phát biểu. Bằng cách bắt buộc mọi claim phải có citation hợp lệ và kiểm tra citation đó ở tầng Core (không do LLM tự xác nhận), hệ thống đảm bảo rằng nội dung xuất hiện trong câu trả lời cuối cùng luôn có thể truy vết ngược về đúng nguồn evidence đã được phê duyệt trong corpus, thay vì chấp nhận nội dung chỉ vì nó "nghe có vẻ đúng" hay trình bày kèm một citation không được xác thực.



Hình 3: Cơ chế kiểm citation: đối chiếu câu trả lời của LLM với evidence pack trước khi render

Như minh hoạ ở Hình 3, bước kiểm citation diễn ra hoàn toàn ở tầng Core — tách biệt khỏi LLM sinh câu trả lời — nhằm đảm bảo quyết định "hợp lệ hay không" không phụ thuộc vào chính mô hình có khả năng sinh ra nội dung sai.

Trường hợp câu trả lời chứa citation không hợp lệ, hoặc không có evidence phù hợp nào được tìm thấy trong corpus ngay từ bước retrieval, hệ thống ưu tiên trả về trạng thái `not_found` thay vì hiển thị một câu trả lời thiếu căn cứ — nhất quán với nguyên tắc an toàn thông tin được áp dụng xuyên suốt toàn hệ thống.

5.7 Kiểm soát chất lượng tài liệu (Coverage)

Bên cạnh các cơ chế kiểm soát trong lúc xử lý câu hỏi (retrieval, citation validation — Mục ?? và ??), US02 còn có một lớp kiểm soát vận hành ngoài luồng (offline), diễn ra *trước khi* người dùng đặt câu hỏi: kiểm kê và đối chiếu độ phủ (coverage) giữa giao diện thực tế và corpus đã có. Đây là cơ chế nhằm phát hiện sớm tình trạng UI thay đổi nhưng tài liệu giải thích chưa được cập nhật theo kịp.

5.7.1 *ui-inventory.json* và *gap-report.json*

Thư mục `binding/unis/catalog/coverage` chứa hai artifact phục vụ việc kiểm kê này:

- ***ui-inventory.json*** — bản kiểm kê toàn bộ nhãn được quét tự động từ frontend của hệ thống SCP. Đây là "ảnh chụp" đầy đủ của mọi nhãn, trạng thái, nút, filter và chỉ số hiển thị đang thực sự tồn tại trên giao diện tại thời điểm quét, không phụ thuộc vào việc corpus đã có tài liệu cho nhãn đó hay chưa.
- ***gap-report.json*** — báo cáo chênh lệch (gap), liệt kê những nhãn xuất hiện trong *ui-inventory.json* nhưng *chưa* có UI concept tương ứng trong corpus, và cũng chưa có *exclusion* (loại trừ) đã được duyệt để xác nhận là không cần giải thích.

Nguyên tắc vận hành ở đây là: mỗi nhãn UI được quét ra phải rơi vào đúng một trong hai trạng thái — hoặc đã có UI concept để US02 có thể giải thích, hoặc đã có exclusion xác nhận rằng nhãn đó không cần giải thích (ví dụ: nhãn thuần trang trí, không mang ý nghĩa nghiệp vụ). Nhãn nào không thuộc trạng thái nào trong hai trạng thái trên sẽ xuất hiện trong *gap-report.json* như một khoảng trống cần xử lý.

Cần phân biệt rõ vai trò của hai artifact này với corpus runtime đã trình bày ở Mục ??: *ui-inventory.json* và *gap-report.json* là công cụ *kiểm soát chất lượng ở tầng vận hành/bảo trì* (quality gate), phục vụ đội ngũ quản trị nội dung phát hiện khoảng trống tài liệu.

Chúng không phải là nguồn dữ liệu mà LLM sử dụng trực tiếp khi trả lời câu hỏi trong thời gian thực — vai trò đó thuộc về năm nhóm dữ liệu trong corpus runtime (Evidence backend, Formula, Rule, Relation, UI semantic).

5.7.2 Coverage là quality gate, không phải đảm bảo độ chính xác

Một điểm cần đặc biệt lưu ý khi diễn giải kết quả coverage: đạt được độ phủ 100

Coverage chỉ trả lời được câu hỏi *"nhân này đã có tài liệu giải thích hay chưa"*, chứ không trả lời được câu hỏi *"nội dung giải thích đó có còn đúng với logic hệ thống hiện tại hay không"*. Có ít nhất ba lý do khiến hai câu hỏi này không tương đương:

- **Nội dung có thể lỗi thời.** Một UI concept từng được duyệt là chính xác tại thời điểm tạo, nhưng nếu logic nghiệp vụ hoặc công thức tính toán phía sau thay đổi mà corpus chưa được cập nhật, US02 vẫn sẽ trả lời dựa trên nội dung cũ — đây chính là loại lỗi mà nhãn phản hồi `huong_dan_loi_thoi` (Mục 5.8) được thiết kế để phát hiện.
- **Coverage đo ở mức nhãn, không đo ở mức câu hỏi thực tế.** Một nhãn có thể đã có UI concept, nhưng cách người dùng thực sự đặt câu hỏi về nhãn đó (từ ngữ, ngữ cảnh, cách diễn đạt) có thể không được retrieval nhận diện đúng — đây là lỗi thuộc tầng retrieval/synthesis, không phải lỗi thiếu coverage.
- **Việc quét UI có độ trễ tự nhiên với thay đổi thực tế.** `ui-inventory.json` phản ánh trạng thái giao diện tại thời điểm quét gần nhất; giữa hai lần quét, nếu giao diện được cập nhật, sẽ tồn tại một khoảng thời gian mà coverage báo cáo không phản ánh đúng hiện trạng.

Vì vậy, coverage nên được hiểu đúng vai trò của nó: một ngưỡng điều kiện cần (quality gate) để đảm bảo không có nhãn UI nào bị bỏ sót hoàn toàn, chứ không phải một điều kiện đủ để khẳng định chất lượng nội dung. Để đánh giá được điều kiện đủ đó, hệ thống cần bổ sung các cơ chế đánh giá khác mang tính liên tục: đánh giá theo bộ dữ liệu chuẩn (*golden evaluation*) được xây dựng và cập nhật định kỳ, kết hợp với phản hồi thực tế từ người dùng qua audit và feedback (Mục 5.8). Nói cách khác, coverage trả lời câu hỏi *"đã nói chưa"*, còn golden evaluation và feedback thực tế mới trả lời được câu hỏi *"nói có đúng không"* — hai lớp kiểm soát này cần được vận hành song song, không thể thay thế cho nhau.

5.8 Vận hành và chi phí

Việc US02 grounded bằng evidence và có kiểm soát citation (Mục ??) không đến miễn phí: so với cách tiếp cận catalog tĩnh trước đây, mỗi lượt xử lý của US02 tiêu tốn nhiều lời gọi LLM hơn, kéo

theo chi phí về độ trễ (latency) và token. Đồng thời, để đánh giá được chất lượng vận hành thực tế — thay vì chỉ dựa vào thiết kế lý thuyết — hệ thống cần một cơ chế ghi nhận và phản hồi liên tục. Mục này trình bày hai khía cạnh đó: chi phí tính theo số lời gọi LLM, và cơ chế audit/feedback phục vụ giám sát chất lượng.

5.8.1 Số lời gọi LLM mỗi lượt

Một lượt xử lý US02 đầy đủ thường bao gồm ba lời gọi LLM tương ứng với ba tác vụ khác nhau trong pipeline (Hình 2), được tóm tắt trong Bảng 5.

Bảng 5: Ba lời gọi LLM trong một lượt xử lý US02

Thứ tự	Tác vụ	Vai trò
1	light (tool selection)	Chọn tool phù hợp (US01/US02/US03) và trích entity/module ban đầu
2	us02_planning	Planner tạo EvidencePlan: xác định intent, module, entity, field, evidence kind, keyword cần truy hồi
3	us02_synthesis	Tổng hợp câu trả lời có citation từ evidence pack

Cần lưu ý rằng lời gọi thứ nhất (light) là bước dùng chung cho cả ba luồng US01/US02/US03, không phải chi phí riêng của US02. Tuy nhiên, hai lời gọi tiếp theo (us02_planning và us02_synthesis) là đặc thù của US02 — đây là điểm khác biệt so với US01 và US03, vốn chỉ cần một lời gọi synthesis duy nhất sau bước truy hồi/thực thi.

Việc tách planner và synthesis thành hai lời gọi riêng biệt (thay vì gộp chung) là đánh đổi có chủ đích: planner chỉ tập trung vào việc xác định phạm vi truy hồi chính xác (intent, module, field...), trong khi synthesis chỉ tập trung vào việc diễn giải evidence đã có — giúp mỗi lời gọi có phạm vi trách nhiệm hẹp và rõ ràng hơn, dễ kiểm soát chất lượng đầu ra hơn so với việc gộp cả hai tác vụ vào một lời gọi duy nhất. Đổi lại, số lời gọi LLM trên mỗi câu hỏi tăng lên, kéo theo độ trễ tổng thể và chi phí token cao hơn so với kiến trúc catalog tĩnh trước đây.

Vì lý do trên, độ trễ (latency) và mức tiêu thụ token của US02 là chỉ số cần được theo dõi liên tục trong vận hành thực tế, đặc biệt khi lưu lượng câu hỏi tăng hoặc khi nhiều luồng (US01/US02/US03) được kích hoạt song song cho cùng một câu hỏi lai (Mục 1.2). Ngoài ba lời gọi LLM nêu trên, bước embedding của BGE-M3 phục vụ retrieval chạy nội bộ trên CPU và không tiêu tốn quota API LLM, nên không được tính vào chi phí ở bảng trên.

5.8.2 Audit và feedback

Mỗi lượt xử lý của US02 được ghi lại đầy đủ vào audit log, bao gồm: caller, identity, câu hỏi, tool được chọn, module/route, token usage, evidence/tài liệu đã gửi sang model, citation, trạng thái xử lý và lỗi (nếu có) — các trường dữ liệu nhạy cảm trong log đã được che thông tin trước khi lưu trữ. Audit log là nguồn dữ liệu quan trọng để đánh giá vận hành thực tế, bổ sung cho thiết kế lý thuyết đã trình bày ở các mục trước: ví dụ, tỷ lệ các trạng thái `not_found/need_clarification/out_of_scope` theo thời gian phản ánh mức độ phù hợp giữa corpus hiện có và nhu cầu thực tế của người dùng, hoặc tần suất `degraded_channels` xuất hiện (Mục 4.4) cho biết mức độ ổn định của hạ tầng vector.

Song song với audit, hệ thống thu thập phản hồi trực tiếp từ người dùng sau mỗi câu trả lời, phân theo ba loại:

- `dung` — câu trả lời chính xác;
- `sai` — câu trả lời không chính xác;
- `huong_dan_loi_thoi` — nội dung từng đúng nhưng nay đã lỗi thời so với hệ thống hiện tại.

Việc phân tách ba loại phản hồi này — thay vì chỉ dùng cơ chế "đúng/sai" nhị phân — có giá trị vận hành quan trọng: nó giúp đội vận hành phân biệt được *nguyên nhân gốc* của một câu trả lời không đạt. Phản hồi `sai` thường gợi ý vấn đề ở tầng retrieval hoặc synthesis (chọn sai evidence, diễn giải sai evidence), cần điều chỉnh mô hình hoặc cấu hình truy hồi. Ngược lại, phản hồi `huong_dan_loi_thoi` không phản ánh lỗi của mô hình, mà cho biết corpus cần được cập nhật để theo kịp thay đổi của hệ thống — tức là vấn đề thuộc về quy trình bảo trì nội dung (Mục 6), không phải lỗi kỹ thuật của pipeline retrieval/synthesis.

Kết hợp audit log và feedback có phân loại tạo thành một vòng lặp giám sát chất lượng liên tục cho US02: audit cung cấp bức tranh định lượng về hành vi hệ thống (tần suất lỗi, chi phí, độ trễ), còn feedback cung cấp tín hiệu định tính từ chính người dùng cuối để xác định nên ưu tiên cải thiện retrieval/synthesis hay cập nhật corpus. Đây là cơ chế cần thiết để bù đắp cho giới hạn đã nêu ở Mục 6: coverage đầy đủ về mặt hình thức không đồng nghĩa với nội dung luôn chính xác, và chỉ có dữ liệu vận hành thực tế mới phát hiện được khoảng cách này.

Phần 6. PHÂN TÍCH LUỒNG US03 - TRUY VẤN SỐ LIỆU THỰC TẾ

6.1 Mục đích, phạm vi và đặc trưng của US03

US03 (định danh công cụ: `query_data`) là luồng chuyên trách xử lý các truy vấn yêu cầu số liệu thực tế đang tồn tại trong cơ sở dữ liệu SCP. Nếu US01 tập trung vào việc hướng dẫn người dùng thực hiện thao tác trên giao diện và US02 giải thích ý nghĩa của trường dữ liệu, công thức hoặc quy tắc nghiệp vụ, thì US03 giải quyết nhóm câu hỏi mang tính định lượng hoặc yêu cầu truy xuất trực tiếp dữ liệu đang được hệ thống lưu trữ. Các dạng truy vấn điển hình có thể bao gồm: “*Tổng đơn kéo hàng đã nhận là bao nhiêu?*”, “*Tổng m² đơn kéo hàng BNC Bắc Ninh tháng này?*”, “*Tổng tiền đơn kéo chưa VAT là bao nhiêu?*” hoặc “*Liệt kê đơn B2B theo trạng thái.*”.

Về bản chất, US03 giải quyết bài toán chuyển đổi truy vấn ngôn ngữ tự nhiên sang truy vấn dữ liệu có cấu trúc, thường được biết đến dưới tên gọi *Text-to-SQL*. Tuy nhiên, khác với mô hình *Text-to-SQL* truyền thống, nơi Mô hình ngôn ngữ lớn (LLM) có thể được cung cấp trực tiếp schema cơ sở dữ liệu và tự do sinh câu lệnh SQL, US03 được xây dựng theo mô hình **Controlled Text-to-SQL**. Trong kiến trúc này, LLM chỉ đảm nhận một số nhiệm vụ có tính suy luận như nhận diện đối tượng dữ liệu liên quan, lựa chọn semantic view và đề xuất kế hoạch truy vấn. Quyền quyết định một câu SQL có được phép thực thi hay không vẫn thuộc về các thành phần kiểm tra tất định của Assistant Core.

Điểm khác biệt trên đặc biệt quan trọng đối với hệ thống SCP, bởi một câu SQL hợp lệ về mặt cú pháp chưa chắc đã đúng về mặt nghiệp vụ. Ví dụ, cụm từ “*tổng tiền*” có thể biểu diễn nhiều đại lượng khác nhau như tổng tiền trước VAT, tổng tiền sau VAT, giá trị bán, giá trị net hoặc giá trị của một báo cáo cụ thể. Tương tự, khái niệm “*số lượng*” có thể được hiểu là số đơn hàng, số dòng dữ liệu, khối lượng tính theo kilogram, số pallet hoặc diện tích tính theo mét vuông. Nếu chỉ để LLM lựa chọn một cột có tên gần nghĩa và áp dụng hàm `SUM()`, câu truy vấn vẫn có thể thực thi thành công nhưng kết quả trả về lại sai hoàn toàn về ý nghĩa nghiệp vụ.

Do đó, mục tiêu của US03 không đơn thuần là *sinh được SQL*, mà là xây dựng một chuỗi xử lý đảm bảo đồng thời ba yêu cầu: **đúng ý nghĩa nghiệp vụ**, **an toàn khi thực thi** và **có khả năng truy vết**. Một truy vấn chỉ được coi là thành công khi hệ thống xác định đúng metric cần sử dụng, lựa chọn đúng nguồn dữ liệu, xây dựng câu SQL phù hợp với câu hỏi, vượt qua các rào chắn an toàn, thực thi thành công trên cơ sở dữ liệu ở chế độ chỉ đọc và cuối cùng trả lại kết quả có đầy đủ metadata về đơn vị và phạm vi dữ liệu.

1. Phạm vi nghiệp vụ của US03

US03 phục vụ các câu hỏi cần truy xuất giá trị hoặc danh sách từ dữ liệu đang tồn tại trong hệ thống. Có thể chia phạm vi này thành một số nhóm chính như sau:

- **Truy vấn đếm:** Xác định số lượng bản ghi hoặc thực thể thỏa mãn một điều kiện nhất định, ví dụ số đơn hàng đang chờ duyệt hoặc số đơn phát sinh trong một khoảng thời gian.
- **Truy vấn tổng hợp:** Tính toán các metric đã được định nghĩa trước như tổng số lượng, tổng diện tích, tổng khối lượng hoặc tổng giá trị tiền.
- **Truy vấn danh sách:** Trả về tập bản ghi thỏa mãn các điều kiện về trạng thái, thời gian, chi nhánh, đối tượng hoặc các chiều dữ liệu khác.
- **Truy vấn thống kê có điều kiện:** Kết hợp các phép lọc và tổng hợp để cung cấp số liệu theo một phạm vi nghiệp vụ cụ thể, chẳng hạn theo tháng, quý, trạng thái hoặc đơn vị tổ chức.

Phạm vi của US03 không bị giới hạn vào một tập câu hỏi mẫu cố định. Các bộ dữ liệu kiểm thử như `golden*.jsonl` chỉ được sử dụng cho mục đích đánh giá và kiểm thử hồi quy, không phải là danh sách đầy đủ những câu hỏi mà hệ thống có thể trả lời. Trong quá trình vận hành thực tế, một câu hỏi mới vẫn có thể được xử lý nếu ý nghĩa của nó có thể biểu diễn thông qua các semantic view, cột dữ liệu, metric, dimension và đường join đã được khai báo và phê duyệt trong Binding Pack.

Điều này tạo ra sự cân bằng giữa tính linh hoạt và tính kiểm soát. Người dùng không bị giới hạn ở một tập câu hỏi được viết sẵn, trong khi LLM cũng không có quyền tự mở rộng phạm vi dữ liệu hoặc tự tạo ra công thức mới. Khả năng của US03 vì vậy được quyết định bởi semantic contract của hệ thống thay vì bởi tri thức nền hoặc khả năng suy luận tự do của LLM.

2. Ranh giới với US01 và US02

Mặc dù cả ba luồng US01, US02 và US03 đều có thể hoạt động trên cùng một module nghiệp vụ, sự phân chia giữa chúng được xác định dựa trên *bản chất yêu cầu của người dùng*. Cùng một khái niệm có thể dẫn tới các luồng xử lý hoàn toàn khác nhau.

Ví dụ, với khái niệm “giá bán buôn”:

- Câu hỏi “*Giá bán buôn được tính thế nào?*” yêu cầu giải thích công thức và được xử lý bởi US02.
- Câu hỏi “*Làm sao tạo giá bán buôn?*” yêu cầu hướng dẫn thao tác và được xử lý bởi US01.
- Câu hỏi “*Giá bán buôn của đơn này là bao nhiêu?*” yêu cầu lấy giá trị thực tế từ dữ liệu và được xử lý bởi US03.

Như vậy, US03 không chịu trách nhiệm giải thích vì sao một metric có công thức như vậy hoặc hướng dẫn người dùng thao tác để tạo dữ liệu. Nhiệm vụ của luồng này bắt đầu tại thời điểm cần lấy giá trị thực tế của metric hoặc bản ghi đang tồn tại trong hệ thống.

Đối với các câu hỏi phức hợp, một truy vấn có thể đồng thời kích hoạt nhiều luồng. Ví dụ:

“Tháng này có bao nhiêu đơn chờ duyệt và Chờ duyệt nghĩa là gì?”

Trong trường hợp trên, phần yêu cầu “có bao nhiêu đơn” được chuyển tới US03 để truy vấn cơ sở dữ liệu, trong khi phần “Chờ duyệt nghĩa là gì” được chuyển tới US02 để giải thích ý nghĩa trạng thái. Hai luồng có thể được thực thi song song và kết quả được tổng hợp trước khi trả về người dùng. Thiết kế này cho phép mỗi phân hệ chỉ đảm nhiệm đúng chức năng chuyên môn của mình thay vì yêu cầu một LLM duy nhất xử lý toàn bộ câu hỏi từ đầu đến cuối.

3. Phạm vi dữ liệu được phép truy cập

Một nguyên tắc quan trọng của US03 là LLM không được truy vấn trực tiếp các bảng nghiệp vụ trong schema **public**. Thay vào đó, dữ liệu phải được truy cập thông qua một tầng ngữ nghĩa riêng được triển khai trong schema **assistant**. Tầng này chỉ bộc lộ những view, cột, phép join và metric đã được đánh giá và phê duyệt trước.

Luồng truy cập dữ liệu về mặt logic có thể biểu diễn như sau:

User Question → Assistant Core → US03 → Semantic Layer → Database
SCP

thay vì:

LLM → Business Tables

Việc bổ sung Semantic Layer tạo ra một ranh giới cách ly giữa mô hình sinh ngôn ngữ và cơ sở dữ liệu nghiệp vụ. Khi cấu trúc bảng nguồn thay đổi hoặc khi một trường dữ liệu nhạy cảm không được phép cung cấp cho chatbot, các thay đổi có thể được kiểm soát tại tầng semantic mà không cần trao quyền truy cập trực tiếp cho mô hình.

Bên cạnh Semantic Layer, phạm vi hoạt động của US03 còn được xác định bởi *Binding Pack*. Đây là tập hợp các cấu hình mô tả những thành phần dữ liệu được phép sử dụng, bao gồm danh sách bảng và cột, semantic view, metric, các quy tắc join, ví dụ few-shot và các trường hợp kiểm thử. Vì vậy, khả năng truy vấn của US03 luôn được giới hạn bởi một tập hợp dữ liệu và công thức hữu hạn đã được kiểm duyệt trước.

4. Nguyên tắc không tự suy diễn số liệu

Một đặc trưng quan trọng khác của US03 là nguyên tắc không suy diễn metric ngoài hợp đồng đã được phê duyệt. Nếu câu hỏi của người dùng yêu cầu một đại lượng chưa được định

nghĩa trong Metric Registry, hệ thống không được tự tạo công thức chỉ vì có thể tìm thấy các cột dữ liệu tương ứng.

Ví dụ, việc cơ sở dữ liệu tồn tại hai cột số không đồng nghĩa với việc LLM được phép tự suy luận:

```
1 SELECT SUM(column_a * column_b)
2 FROM assistant.some_view;
```

và coi biểu thức này là một metric nghiệp vụ hợp lệ. Một phép tính chỉ được sử dụng khi ý nghĩa, công thức, nguồn dữ liệu, grain, output và đơn vị của nó đã được khai báo trong hợp đồng metric tương ứng.

Nguyên tắc này giúp tách biệt hai khái niệm thường bị nhầm lẫn trong các hệ thống Text-to-SQL:

- Khả năng biểu diễn bằng SQL: một phép tính có thể viết được dưới dạng câu lệnh SQL;
- Tính hợp lệ nghiệp vụ: phép tính đó có thực sự phản ánh đúng định nghĩa của hệ thống hay không.

US03 chỉ cho phép thực thi khi đồng thời thỏa mãn cả hai điều kiện.

5. Nguyên tắc Read-only và Bounded Execution

US03 được thiết kế theo mô hình *read-only*. Trong quá trình vận hành, Assistant Core chỉ sử dụng tài khoản cơ sở dữ liệu có quyền đọc trên semantic schema. Luồng không được phép thực hiện các thao tác thay đổi trạng thái hệ thống như INSERT, UPDATE, DELETE, DROP, ALTER hoặc các câu lệnh DDL/DML khác.

Ngoài giới hạn về quyền truy cập, mỗi câu truy vấn còn phải tuân thủ nguyên tắc *Bounded Execution*. Một câu SQL do LLM sinh ra không được thực thi trực tiếp mà phải đi qua nhiều lớp kiểm soát độc lập. Chuỗi kiểm soát tổng quát bao gồm:

SqlPlan → Business Contract Validation → AST Guardrail → EXPLAIN →
Read-only Execution

Các lớp này kiểm soát những loại rủi ro khác nhau. Contract Validation đảm bảo truy vấn đúng về ý nghĩa nghiệp vụ; AST Guardrail đảm bảo cấu trúc SQL không chứa các thao tác bị cấm; EXPLAIN kiểm tra chi phí trước khi thực thi; trong khi quyền *read-only* của cơ sở dữ liệu đóng vai trò là lớp bảo vệ cuối cùng nếu các tầng phía trên không phát hiện được một trường hợp bất thường.

Có thể xem đây là kiến trúc *defense-in-depth*, trong đó an toàn của US03 không phụ thuộc vào một rào chắn duy nhất mà được hình thành bởi nhiều tầng kiểm soát độc lập:

Metric Contract → Semantic Layer → Business Validation → SQL
Guardrail → Cost Guardrail → Database Permission

6. Nguyên tắc Fail-safe và tính trung thực của kết quả

US03 tuân thủ nguyên tắc *fail-safe*: khi hệ thống không thể xác định hoặc thực thi một truy vấn một cách đáng tin cậy, luồng phải dừng lại và trả về trạng thái lỗi phù hợp thay vì cố gắng tạo ra một con số.

Đặc biệt, cần phân biệt rõ giữa hai trường hợp:

Không có dữ liệu $\neq$ Không truy vấn được dữ liệu.

Nếu một truy vấn hợp lệ thực thi thành công và không có bản ghi phù hợp, giá trị 0 hoặc tập kết quả rỗng có thể là một kết quả nghiệp vụ hợp lệ. Ngược lại, nếu hệ thống không sinh được SQL, truy vấn bị guardrail từ chối hoặc database xảy ra lỗi, US03 không được trả về 0 như một giá trị thay thế. Việc làm như vậy sẽ biến một lỗi kỹ thuật thành một kết luận nghiệp vụ sai và đặc biệt nguy hiểm trong các báo cáo quản trị.

Do đó, US03 sử dụng các trạng thái kết thúc riêng như `need_clarification`, `out_of_scope`, `insufficient_schema` và `failed` để biểu diễn chính xác nguyên nhân mà một câu hỏi không thể trả về số liệu.

7. Khả năng truy vết và kiểm chứng

Mỗi truy vấn US03 không chỉ tạo ra kết quả cuối cùng mà còn để lại dấu vết phục vụ kiểm tra và đánh giá. Hệ thống ghi nhận các thông tin quan trọng như người gửi yêu cầu, câu hỏi gốc, SQL được mô hình đề xuất, SQL sau guardrail, chi phí ước lượng, số dòng kết quả, thời gian thực thi và trạng thái của truy vấn.

Khả năng truy vết này đặc biệt cần thiết đối với số liệu nghiệp vụ. Khi một người dùng phản ánh rằng kết quả chatbot khác với báo cáo SCP, đội vận hành có thể lần ngược theo chuỗi:

Question → Metric → Semantic View → SQL → Filters → Result

để xác định sai lệch xuất phát từ bước nhận diện ý nghĩa câu hỏi, lựa chọn metric, xây dựng SQL, phạm vi dữ liệu hay dữ liệu nguồn.

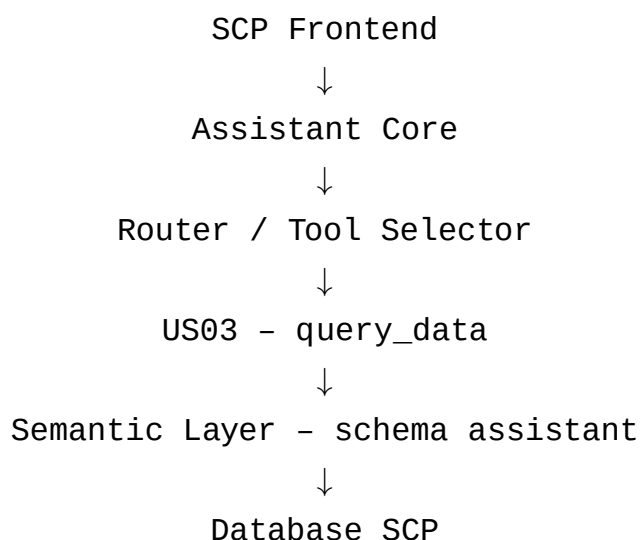
Từ các đặc trưng trên có thể thấy rằng US03 không được thiết kế như một chatbot có khả năng “hỏi trực tiếp cơ sở dữ liệu”, mà là một pipeline truy vấn dữ liệu có kiểm soát. LLM được sử dụng để xử lý những phần cần khả năng hiểu ngôn ngữ tự nhiên và suy luận linh hoạt, trong khi mọi quyết định ảnh hưởng trực tiếp đến tính đúng đắn và an toàn của dữ liệu đều được bao quanh bởi các hợp đồng và rào chắn chặt chẽ. Đây là nền tảng để US03 vừa duy trì khả năng xử lý các cách diễn đạt đa dạng của người dùng, vừa đảm bảo rằng kết quả cuối cùng luôn nằm trong phạm vi dữ liệu và logic nghiệp vụ đã được hệ thống phê duyệt. “

“‘latex

6.2 Kiến trúc dữ liệu phục vụ US03

Khác với US01 và US02, vốn chủ yếu làm việc với tài liệu hướng dẫn và tập evidence trong Knowledge Base, US03 phải tương tác với dữ liệu nghiệp vụ đang thay đổi theo thời gian trong cơ sở dữ liệu SCP. Vì vậy, kiến trúc dữ liệu của US03 được thiết kế theo nguyên tắc *tách biệt nguồn dữ liệu nghiệp vụ khỏi không gian mà LLM được phép quan sát và truy vấn*. LLM không nhìn thấy trực tiếp toàn bộ cấu trúc cơ sở dữ liệu nguồn mà chỉ làm việc với một tập semantic view, cột, metric và quan hệ đã được kiểm duyệt trước.

Có thể mô hình hóa kiến trúc dữ liệu của US03 theo ba lớp 395(v)24-f6.2



Trong luồng này, Router chịu trách nhiệm xác định rằng câu hỏi của người dùng thuộc nhóm truy vấn số liệu thực tế. Sau khi `query_data` được chọn, US03 không trực tiếp gửi câu hỏi tới database mà khởi tạo một chuỗi xử lý riêng gồm nhận diện metric, lựa chọn semantic view, sinh `SQLPlan`, kiểm tra hợp đồng nghiệp vụ, kiểm tra an toàn SQL, đánh giá chi phí truy vấn và cuối cùng mới thực thi câu lệnh ở chế độ chỉ đọc.

Một điểm quan trọng trong kiến trúc là US03 được triển khai như một công cụ độc lập về mặt runtime. Công cụ này chỉ được coi là sẵn sàng khi Assistant Core có thể kết nối tới semantic schema của SCP, có database runner phù hợp và có audit sink hoạt động. Nếu một trong các phụ thuộc cần thiết chưa sẵn sàng, `query_data` không được đưa vào tập công cụ khả dụng. Tuy nhiên, việc US03 chưa sẵn sàng không làm ngừng hoạt động của US01 hoặc US02. Cơ chế này bảo đảm tính cách ly giữa các phân hệ và tránh trường hợp một lỗi kết nối database kéo theo toàn bộ Assistant Core ngừng phục vụ.

Về mặt trách nhiệm, US03 cũng được tách rõ khỏi hai luồng còn lại:

- US01 sử dụng Knowledge Base để truy hồi tài liệu thao tác;
- US02 sử dụng corpus evidence để giải thích logic, công thức và ý nghĩa hệ thống;
- US03 sử dụng Semantic Layer để truy vấn dữ liệu thực tế đang tồn tại trong SCP.

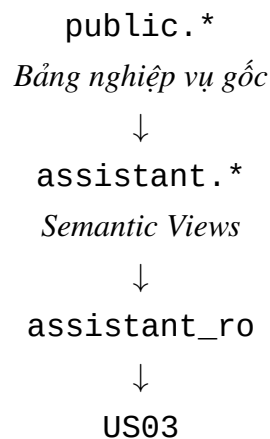
Sự tách biệt này giúp mỗi luồng có thể áp dụng một mô hình kiểm soát riêng. Đối với US03, yếu tố quan trọng nhất không phải là retrieval hay citation mà là *correctness của truy vấn, quyền truy cập dữ liệu và an toàn khi thực thi SQL*. Chính vì vậy, US03 có thêm các thành phần đặc thù như Metric Registry, SQL Contract Validation, AST Guardrail, EXPLAIN Cost Check và tài khoản database assistant_ro.

Có thể xem US03 là một “cổng truy vấn dữ liệu có kiểm soát” nằm giữa Assistant Core và dữ liệu nghiệp vụ. LLM chỉ đóng vai trò đề xuất cách truy vấn, trong khi Core quyết định phạm vi dữ liệu, cấu trúc SQL được phép sử dụng và điều kiện cần thiết trước khi cho phép truy vấn chạm tới database.

6.2.2 Tầng ngữ nghĩa *Semantic Layer*

Tầng ngữ nghĩa (*Semantic Layer*) là thành phần trung gian quan trọng nhất giữa US03 và cơ sở dữ liệu SCP. Thay vì cấp quyền truy cập trực tiếp vào các bảng nghiệp vụ trong schema `public`, hệ thống xây dựng một schema riêng có tên `assistant`, bên trong chứa các view được thiết kế dành riêng cho nhu cầu truy vấn của chatbot.

Mô hình truy cập dữ liệu có thể mô tả như sau:



Semantic Layer thực hiện đồng thời nhiều vai trò.

Thứ nhất, cách ly cấu trúc cơ sở dữ liệu nghiệp vụ. LLM không cần biết và không được phép biết toàn bộ cấu trúc bảng nguồn. Những bảng chứa các trường kỹ thuật, dữ liệu nhạy cảm hoặc các quan hệ phức tạp không cần thiết sẽ không được expose ra schema `assistant`. Điều này làm giảm đáng kể không gian mà LLM có thể sử dụng khi sinh SQL.

Thứ hai, chuẩn hóa ngữ nghĩa nghiệp vụ. Một bảng nguồn có thể được thiết kế để tối ưu cho giao dịch hoặc hiệu năng của backend, nhưng cấu trúc đó không nhất thiết thuận tiện cho truy vấn bằng ngôn ngữ tự nhiên. Semantic view có thể đổi tên trường, chuẩn hóa kiểu dữ liệu, gom nhóm một số logic nghiệp vụ hoặc thể hiện một grain phù hợp hơn với câu hỏi của người dùng. Nhờ đó, schema được cung cấp cho LLM gần với ngôn ngữ nghiệp vụ hơn so với schema vật lý của hệ thống.

Thứ ba, cố định phạm vi truy vấn được phê duyệt. Mọi bảng mà SQL của US03 tham chiếu phải thuộc schema `assistant` và nằm trong whitelist runtime. Do đó, ngay cả khi LLM sinh ra

tên một bảng có thật trong database nhưng bảng đó không được expose qua Semantic Layer, truy vấn vẫn bị chặn trước khi thực thi.

Thứ tư, giảm rủi ro sai lệch phép gộp dữ liệu. Nhiều semantic view được xây dựng để thể hiện rõ grain của dữ liệu. Điều này đặc biệt quan trọng với các phép tổng hợp như SUM, COUNT hoặc AVG. Nếu LLM join trực tiếp các bảng ở grain khác nhau, kết quả có thể bị nhân dòng và tạo ra số liệu sai dù câu SQL hoàn toàn hợp lệ về cú pháp. Semantic Layer giúp giảm rủi ro này bằng cách chuẩn hóa cấu trúc đầu vào cho từng nhóm nghiệp vụ.

Thứ năm, tạo ranh giới bảo mật ở cấp database. Role runtime assistant_ro chỉ được cấp quyền SELECT trên schema assistant và không có quyền truy cập trực tiếp các bảng nghiệp vụ trong public. Đây là một lớp bảo vệ độc lập với prompt và SQL guardrail. Ngay cả trong trường hợp một câu SQL bất thường vượt qua được các tầng kiểm tra phía trên, quyền database vẫn giới hạn truy vấn trong vùng dữ liệu đã được cấp phép.

Trong đặc tả hiện tại, phạm vi US03 bao gồm 42 view đã được triển khai trong schema assistant. Các view này được hình thành từ hai nhóm:

- **Projection view:** chủ yếu expose một tập cột được chọn từ bảng nguồn;
- **Semantic view viết tay:** chứa logic nghiệp vụ phức tạp hơn, được sử dụng khi chỉ projection dữ liệu nguồn không đủ để biểu diễn đúng metric hoặc grain cần thiết.

Trong tổng số các view được triển khai, có 10 semantic view viết tay. Toàn bộ schema hiện expose khoảng 449 cột cho US03. Tuy nhiên, các cột này không được đưa đồng thời vào context của LLM trong mỗi lượt truy vấn. Ở bước lựa chọn đối tượng dữ liệu, hệ thống chỉ chọn tối đa một số lượng nhỏ view liên quan, sau đó mới cung cấp schema chi tiết của các view đó cho SQL Generator. Cách tổ chức này vừa giới hạn context, vừa giảm khả năng mô hình chọn nhầm bảng hoặc cột.

Như vậy, Semantic Layer không chỉ là một lớp “view” đơn thuần mà thực chất là một hợp đồng truy cập dữ liệu. Nó xác định dữ liệu nào chatbot được phép nhìn thấy, ở grain nào, dưới tên gọi nào và thông qua nguồn nào.

6.2.3 Binding Pack và phạm vi dữ liệu được phê duyệt

Nếu Semantic Layer xác định dữ liệu thực tế mà US03 có thể truy cập ở cấp database, thì *Binding Pack* xác định cách Assistant Core hiểu và sử dụng tập dữ liệu đó ở cấp runtime. Binding Pack có thể được xem như một bộ đặc tả cấu hình chứa toàn bộ metadata cần thiết để chuyển từ câu hỏi ngôn ngữ tự nhiên sang một truy vấn SQL hợp lệ.

Các thành phần chính của Binding Pack phục vụ US03 được tổng hợp trong Bảng 6.

Bảng 6: Các thành phần chính của Binding Pack phục vụ US03

Thành phần	Vai trò
<code>manifest.yaml</code>	Chứa các tham số runtime như DSN, semantic schema, timeout, giới hạn số dòng và ngưỡng chi phí EXPLAIN.
<code>tables.yaml</code>	Khai báo whitelist các bảng và cột nguồn được phép expose cho US03.
<code>semantic/*.sql</code>	Định nghĩa các semantic view viết tay, phục vụ các công thức hoặc grain nghiệp vụ phức tạp.
<code>metrics/*.yaml</code>	Định nghĩa các metric được phê duyệt: nguồn, grain, measure, output, đơn vị, alias và dimension bắt buộc.
<code>joins.yaml</code>	Mô tả quan hệ giữa các nguồn dữ liệu, cardinality và các đường join có nguy cơ fan-out.
<code>fewshot/us03.jsonl</code>	Chứa các ví dụ câu hỏi-SQL đã được phê duyệt để bổ sung ngữ cảnh cho bước sinh SQL.
<code>modules.yaml</code>	Khai báo danh mục các phân hệ nghiệp vụ được sử dụng chung trong binding.
<code>eval/golden*.jsonl</code>	Chứa các trường hợp kiểm thử nghiệm thu và hồi quy cho US03.

Các file trên không có vai trò ngang nhau. Có thể chia chúng thành ba nhóm chức năng chính.

Nhóm 1 – Hợp đồng dữ liệu: Bao gồm `tables.yaml`, `semantic/*.sql`, `metrics/*.yaml` và `joins.yaml`. Đây là nhóm quyết định trực tiếp truy vấn nào được xem là hợp lệ về mặt dữ liệu và nghiệp vụ.

Nhóm 2 – Cấu hình runtime: `manifest.yaml` xác định cách Core kết nối database và các ngưỡng tài nguyên được áp dụng trong quá trình thực thi, chẳng hạn giới hạn dòng, timeout hoặc ngưỡng chi phí của EXPLAIN.

Nhóm 3 – Hỗ trợ mô hình và đánh giá: `fewshot/us03.jsonl` hỗ trợ LLM sinh SQL chính xác hơn thông qua các ví dụ đã được phê duyệt, trong khi `eval/golden*.jsonl` được sử dụng để kiểm tra độ ổn định của luồng qua các phiên bản.

Một điểm cần phân biệt rõ là **golden set không phải whitelist câu hỏi**. Việc một câu hỏi không xuất hiện trong tập `golden*.jsonl` không có nghĩa US03 không được phép xử lý câu hỏi đó. Golden set chỉ đóng vai trò là tập kiểm thử hồi quy. Trong runtime, một câu hỏi mới vẫn có thể được giải quyết nếu nó biểu diễn được bằng các view, cột, metric, dimension và đường join đã được khai báo trong Binding Pack.

Phạm vi dữ liệu hiện tại của US03 được tổng hợp trong Bảng 7.

Bảng 7: Phạm vi dữ liệu hiện tại của US03

Thành phần	Số lượng
Bảng được khai báo trong <code>tables.yaml</code>	42
View được triển khai trong <code>schema assistant</code>	42
Semantic view viết tay	10
Cột được expose qua các view	449
Metric có thể thực thi	79
Phân hệ nghiệp vụ được bao phủ	8

Tám phân hệ nghiệp vụ nằm trong phạm vi hiện tại gồm Master Data, Supply Sync, Demand Aggregation, Production Lot Sizing, PO Status, Pull Orders, B2B Orders và KHMB.

Các con số trên phản ánh phạm vi mà US03 thực tế có thể truy vấn, không nên nhầm lẫn với catalog dữ liệu của US02. Ví dụ, số lượng bảng xuất hiện trong inventory hoặc catalog dùng cho giải thích hệ thống có thể lớn hơn số bảng được whitelist cho truy vấn số liệu. Hai tập dữ liệu phục vụ hai mục tiêu hoàn toàn khác nhau: US02 cần biết ý nghĩa của hệ thống, trong khi US03 chỉ được phép đọc một tập dữ liệu đã được chọn lọc và kiểm soát chặt hơn.

Một đặc trưng quan trọng khác của Binding Pack là nó tách *quyết định nghiệp vụ* ra khỏi mã sinh SQL của LLM. Khi cần thay đổi một metric, bổ sung một semantic view hoặc cập nhật đường join được phép, đội phát triển có thể cập nhật Binding Pack và triển khai lại semantic layer thay vì sửa prompt để “dạy” LLM cách suy luận mới. Điều này làm tăng tính quản trị và khả năng kiểm soát phiên bản của hệ thống.

6.2.4 Metric Registry và hợp đồng nghiệp vụ

Metric Registry là thành phần trung tâm giúp US03 hiểu một khái niệm định lượng trong câu hỏi người dùng phải được ánh xạ thành biểu thức SQL nào. Trong môi trường SCP, cùng một thuật ngữ tự nhiên có thể mang nhiều nghĩa khác nhau tùy theo module, grain, trạng thái hoặc đơn vị. Vì vậy, việc chỉ dựa vào tên cột trong database không đủ để bảo đảm tính đúng đắn của truy vấn.

Ví dụ, khi người dùng hỏi:

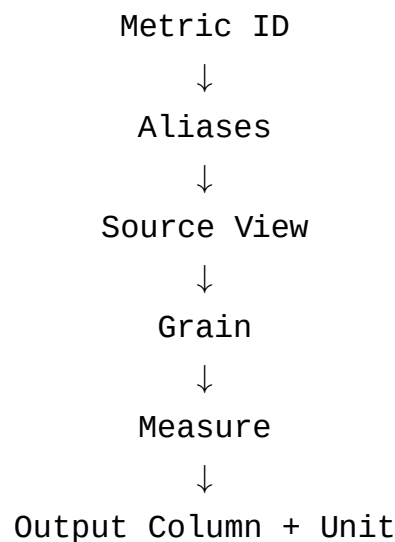
“Tổng tiền đơn kéo là bao nhiêu?”

cụm từ “tổng tiền” chưa chỉ ra duy nhất một công thức. Trong hệ thống có thể tồn tại các đại lượng khác nhau như tổng tiền trước VAT, tổng tiền sau VAT, tiền bán, tiền net hoặc giá trị dùng trong một báo cáo chuyên biệt. Nếu SQL Generator chỉ tìm một cột tên gần với `amount` rồi áp dụng `SUM()`, truy vấn có thể đúng cú pháp nhưng sai hoàn toàn ý nghĩa nghiệp vụ.

Metric Registry giải quyết vấn đề này bằng cách định nghĩa mỗi metric như một hợp đồng có cấu trúc. Một metric điển hình bao gồm các thành phần:

- **metric_id**: định danh duy nhất của metric;
- **Alias**: các cách người dùng có thể diễn đạt metric bằng ngôn ngữ tự nhiên;
- **Source**: semantic view được phép dùng để tính metric;
- **Grain**: mức hạt dữ liệu mà metric được định nghĩa;
- **Measure**: biểu thức tính toán đã được phê duyệt;
- **output_column**: tên cột kết quả bắt buộc;
- **Unit**: đơn vị của metric;
- **required_dimensions**: các chiều bắt buộc phải xuất hiện khi metric yêu cầu;
- **dimension_aliases**: các alias đầu ra của dimension.

Có thể mô hình hóa một metric theo cấu trúc sau:



Một ví dụ trong luồng Pull Orders là metric:

`pull_value_report_total_m2`

Metric này đại diện cho tổng diện tích đơn kéo hàng theo phạm vi báo cáo. Khi câu hỏi chứa cụm “đơn kéo hàng” cùng với dấu hiệu đơn vị “m²”, bộ nhận diện metric sẽ cố gắng khóa câu hỏi vào metric phù hợp thay vì cho SQL Generator tự suy đoán phép tính.

Vai trò của Metric Registry có thể phân tích theo bốn khía cạnh chính.

1. **Khóa ý nghĩa nghiệp vụ trước khi sinh SQL.** Trước khi LLM được yêu cầu sinh truy vấn, US03 chuẩn hóa câu hỏi và đối chiếu với alias của các metric đã được phê duyệt. Metric không được đưa vào tập `required_metrics`. Khi một metric đã trở thành required metric, SQL Generator bắt buộc phải sử dụng metric đó hoặc trả về trạng thái không thể xử lý; không được thay thế bằng một phép tính “gần giống”.

2. **Khóa nguồn dữ liệu.** Metric không chỉ định nghĩa công thức mà còn chỉ rõ semantic view nào là nguồn hợp lệ. Vì vậy, ngay cả khi một cột tương tự tồn tại ở nhiều view khác nhau, LLM không thể tùy ý chọn nguồn khác. Source của required metric được ép vào tập view được chọn ở giai đoạn Object Selection.

3. **Khóa công thức tính.** Phần `measure` trong metric được kiểm tra ở mức AST trước khi SQL chạy. Điều này giúp ngăn một trường hợp phổ biến: LLM khai báo đúng `metric_id` nhưng SQL thực tế lại sử dụng `COUNT(1)`, một cột khác hoặc một biểu thức tổng hợp sai. Core không tin hoàn toàn trường `metrics_used` do LLM khai báo mà kiểm tra trực tiếp biểu thức trong câu SQL.

4. **Khóa định dạng kết quả và đơn vị.** `output_column` và `unit` tạo liên kết giữa SQL và tầng trình bày. Khi query thực thi thành công, Core dựa trên registry để gán `metric_id` và đơn vị vào đúng cột kết quả. Nhờ đó, LLM ở bước synthesis không phải tự đoán xem giá trị trả về là VND, kg, pallet hay m².

Một vấn đề đặc biệt quan trọng là khái niệm **grain**. Grain mô tả một dòng trong nguồn dữ liệu đại diện cho đối tượng nào. Ví dụ, một view có thể ở grain “một dòng trên một đơn hàng”, trong khi view khác ở grain “một dòng trên một chi tiết sản phẩm”. Nếu phép tổng hợp được thực hiện sau khi join hai nguồn ở grain không tương thích, dữ liệu có thể bị nhân dòng.

Ví dụ, giả sử một đơn hàng có giá trị 100 và có ba dòng chi tiết:

		Đơn hàng	Chi tiết
Đơn hàng	Giá trị	A	D1
A	100	A	D2
		A	D3

Sau phép join, giá trị đơn hàng có thể xuất hiện ba lần:

A	100	D1
A	100	D2
A	100	D3

Nếu thực hiện:

```
1 SELECT SUM(order_value)
2 FROM ...;
```

kết quả thu được là 300 thay vì 100. Đây là ví dụ điển hình cho lỗi *fan-out*: SQL đúng cú pháp, truy vấn chạy thành công nhưng số liệu sai do cardinality.

Do đó, Metric Registry không hoạt động độc lập mà kết hợp với `joins.yaml` để xác định grain và các đường join an toàn. Các join được đánh dấu có nguy cơ fan-out sẽ bị kiểm tra hoặc chặn khi query thực hiện phép aggregate. Cách tiếp cận này cho thấy tính đúng đắn trong US03 không được đánh giá chỉ dựa trên SQL syntax mà còn dựa trên *semantic correctness* của toàn bộ đường truy vấn.

Ngoài metric tổng hợp, hệ thống cũng cho phép các truy vấn dạng liệt kê hoặc phân tích theo cột khi không có metric cụ thể khớp câu hỏi. Tuy nhiên, một phép cộng hoặc đại lượng tổng hợp mới không được coi là hợp lệ chỉ vì LLM có khả năng viết được câu lệnh SUM. Nếu đại lượng đó chưa có metric được phê duyệt, US03 không được tự biến nó thành một metric nghiệp vụ mới.

Có thể tóm tắt vai trò của Metric Registry bằng chuỗi sau:

Ngôn ngữ người dùng → Metric Alias → Metric ID → Source + Measure + Grain → SQL Contract → Result + Unit

Nhờ cơ chế này, Metric Registry đóng vai trò như cầu nối giữa ngôn ngữ tự nhiên của người dùng và định nghĩa nghiệp vụ chính thức của hệ thống. Nó giúp US03 giải quyết một trong những vấn đề khó nhất của Text-to-SQL trong môi trường doanh nghiệp: không chỉ tìm ra câu SQL có thể chạy, mà phải bảo đảm câu SQL đó đang trả lời đúng đại lượng mà người dùng thực sự hỏi. “

“‘latex “‘latex

6.3 Kiến trúc tổng thể của luồng xử lý US03

US03 được xây dựng theo kiến trúc xử lý nhiều tầng nhằm chuyển đổi câu hỏi ngôn ngữ tự nhiên của người dùng thành kết quả số liệu có thể kiểm chứng. Khác với cách tiếp cận Text-to-SQL trực tiếp, trong đó mô hình ngôn ngữ sinh câu SQL và gửi ngay tới cơ sở dữ liệu, US03 phân tách toàn bộ quá trình thành nhiều giai đoạn độc lập. Mỗi giai đoạn chịu trách nhiệm cho một nhóm nhiệm vụ cụ thể và đầu ra chỉ được chuyển sang bước tiếp theo khi thỏa mãn các ràng buộc tương ứng.

Kiến trúc này được thiết kế dựa trên nguyên tắc *progressive validation*, nghĩa là truy vấn được kiểm tra từng bước từ cấp độ ý nghĩa nghiệp vụ đến cấu trúc SQL, chi phí thực thi và cuối cùng là quyền truy cập cơ sở dữ liệu. Nhờ đó, một sai lệch xuất hiện ở bước đầu không được truyền trực tiếp tới database và tạo ra kết quả sai.

Toàn bộ luồng US03 có thể được chia thành năm giai đoạn chính:

1. Hiểu truy vấn (Query Understanding);
2. Lập kế hoạch truy vấn (Query Planning);
3. Xác minh truy vấn (Query Validation);
4. Thực thi an toàn (Safe Execution);
5. Trả kết quả (Result Delivery).

Năm giai đoạn trên tạo thành một pipeline liên tục từ lúc người dùng gửi câu hỏi tới khi giao diện nhận được bảng số liệu và phân diễn giải cuối cùng.

6.3.1 Sơ đồ End-to-End

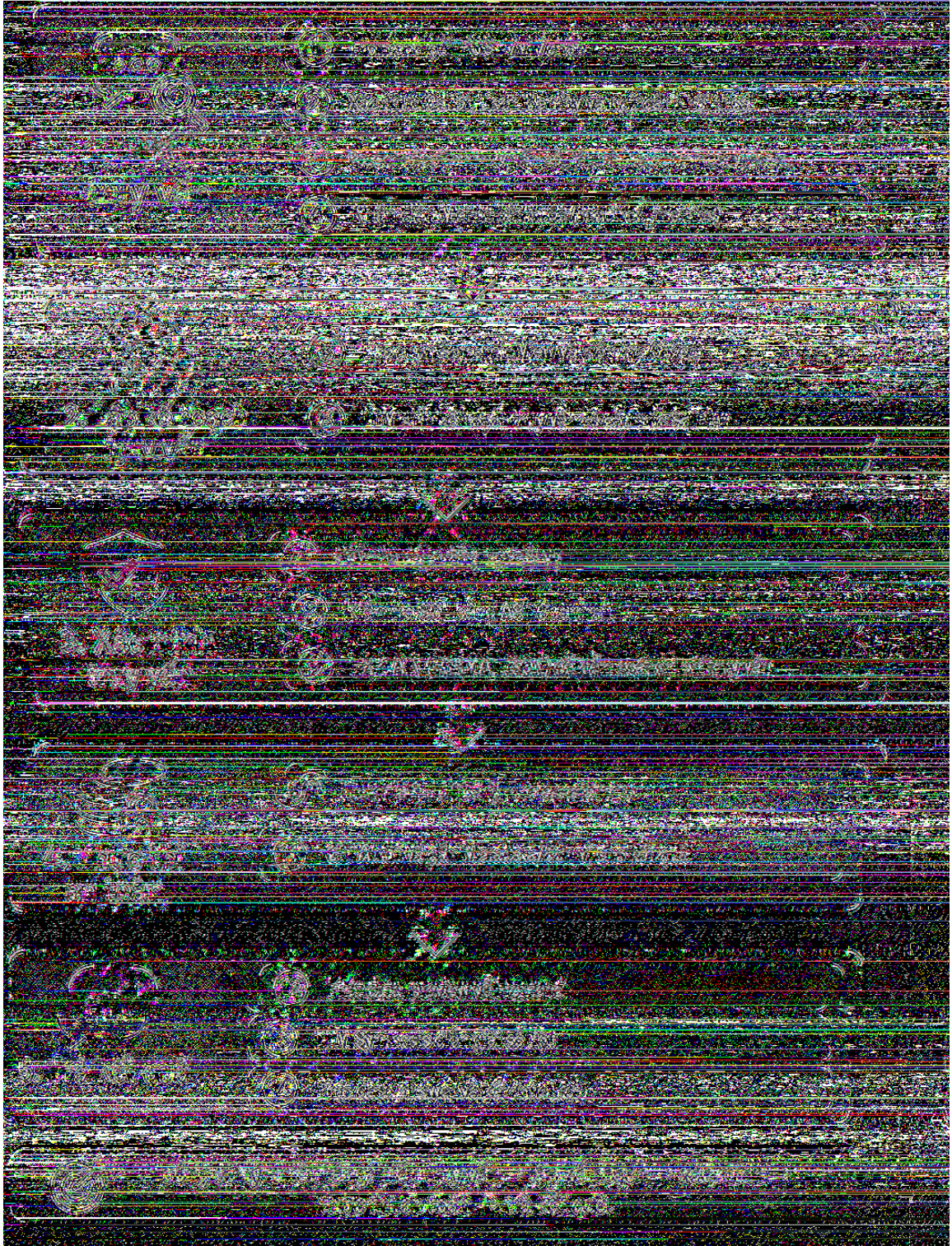
Luồng End-to-End của US03 bắt đầu khi SCP Frontend gửi request tới Assistant Core thông qua API:

POST /v1/chat

Request chứa câu hỏi hiện tại, mã hội thoại và thông tin ngữ cảnh cần thiết. Sau khi xác thực caller và identity, Router xác định câu hỏi có thuộc nhóm truy vấn số liệu thực tế hay không. Nếu có, công cụ query_data được kích hoạt và câu hỏi được chuyển vào luồng xử lý US03.

Hình 4 mô tả toàn bộ chu trình xử lý từ đầu vào đến đầu ra.

Sơ đồ End-to-End của luồng xử lý US03



Hình 4: Sơ đồ End-to-End của luồng xử lý US03

Như thể hiện trong Hình 4, US03 không chuyển trực tiếp câu hỏi thành SQL rồi thực thi ngay. Thay vào đó, mỗi truy vấn phải đi qua một chuỗi các bước được kiểm soát.

Ở giai đoạn đầu tiên, hệ thống xác định đây có phải truy vấn số liệu hay không và nhận diện metric nghiệp vụ mà người dùng đang yêu cầu. Sau đó, US03 lựa chọn các Semantic View phù hợp và sử dụng LLM để tạo một SqlPlan có cấu trúc.

Tuy nhiên, SqlPlan chỉ được xem là một kế hoạch truy vấn đề xuất. Câu SQL bên trong chưa được phép thực thi ngay mà phải lần lượt vượt qua các lớp kiểm tra nghiệp vụ, kiểm tra cấu trúc AST và kiểm tra chi phí bằng EXPLAIN.

Có thể biểu diễn phần lõi của quá trình này như sau:

SqlPlan → Contract Validation → AST Guardrail → EXPLAIN → Execution

Chỉ sau khi vượt qua toàn bộ các bước kiểm tra, câu lệnh SELECT mới được thực thi bằng tài khoản cơ sở dữ liệu assistant_ro. Kết quả sau đó được ghi audit, gắn metadata và đơn vị, rồi phát về frontend thông qua event data_preview. Cuối cùng, LLM thực hiện synthesis để diễn giải dữ liệu trước đó thành một datanswerqlPlan

“Tổng m² đơn kéo hàng chi nhánh BNC Bắc Ninh tháng này.”

hệ thống cần phân tích được các yếu tố như:

- “tổng” thể hiện yêu cầu aggregate;
- “đơn kéo hàng” mô tả đối tượng nghiệp vụ;
- “m²” là dấu hiệu đơn vị;
- “BNC Bắc Ninh” là điều kiện lọc theo chi nhánh;
- “tháng này” là phạm vi thời gian.

Trong đó, bước nhận diện metric có ý nghĩa đặc biệt quan trọng. Hệ thống không để SQL Generator tự suy đoán một phép SUM bất kỳ mà trước hết cố gắng ánh xạ câu hỏi tới metric đã được phê duyệt trong Metric Registry.

Đầu ra chính của giai đoạn này là tập `required_metrics` cùng với ngữ cảnh nghiệp vụ đã được chuẩn hóa.

2. Giai đoạn lập kế hoạch truy vấn – Query Planning

Sau khi hiểu yêu cầu, hệ thống cần xác định nguồn dữ liệu nào có thể trả lời câu hỏi.

Giai đoạn này gồm hai bước chính:

5. Lựa chọn Semantic View và kiểm tra Object;
6. sử dụng LLM để sinh kế hoạch truy vấn SQL plan.

LLM ở tác vụ `light` nhận catalog rút gọn và lựa chọn một số Semantic View có liên quan. Core sau đó bổ sung source bắt buộc của required metric, loại bỏ các object trùng và kiểm tra từng view với schema runtime.

Việc thu hẹp tập view trước khi sinh SQL giúp giảm đáng kể không gian tìm kiếm. Thay vì đưa toàn bộ schema với hàng trăm cột cho LLM, SQL Generator chỉ nhận cấu trúc chi tiết của một số view có khả năng liên quan trực tiếp tới câu hỏi.

Sau khi tập view được xác định, Assistant Core xây dựng prompt dành cho tác vụ `SQL_GENERATION`. Prompt bao gồm:

- các luật SQL tĩnh;

- metric liên quan;
- schema và grain của các view đã chọn;
- thông tin về join;
- few-shot example phù hợp;
- mapping giá trị enum;
- ngày hiện tại;
- câu hỏi người dùng;
- lỗi của lần sinh trước nếu đang thực hiện retry.

Kết quả của LLM không phải chỉ là một câu SQL, mà là một đối tượng SQLPlan có cấu trúc, trong đó mô hình khai báo SQL, các object đã dùng, metric đã dùng và các giả định liên quan.

Điều này tạo điều kiện để Assistant Core kiểm tra kế hoạch một cách có hệ thống trước khi cho phép truy vấn tiến sâu hơn vào pipeline.

3. Giai đoạn xác minh truy vấn – Query Validation

Giai đoạn thứ ba kiểm tra xem truy vấn do LLM tạo ra có thực sự phù hợp với yêu cầu nghiệp vụ và có an toàn về cấu trúc hay không.

Giai đoạn này gồm ba bước:

7. kiểm tra Hợp đồng nghiệp vụ;
8. kiểm tra SQL bằng AST Guardrail;
9. chạy EXPLAIN (FORMAT JSON) để đánh giá chi phí.

Trước hết, Business Contract Validation kiểm tra truy vấn ở mức ngữ nghĩa. Các điều kiện cần kiểm tra bao gồm:

- object có thuộc tập Semantic View được phép hay không;
- metric có tồn tại và sử dụng đúng source hay không;
- required metric có xuất hiện trong metrics_used hay không;
- SQL có sử dụng đúng biểu thức measure hay không;

- tên cột kết quả có đúng `output_column` hay không;
- các dimension bắt buộc có đầy đủ hay không;
- có xuất hiện bộ lọc thời gian mà người dùng không yêu cầu hay không.

Điểm quan trọng là một câu SQL có thể hoàn toàn hợp lệ về mặt cú pháp nhưng vẫn sai về mặt nghiệp vụ. Ví dụ, sử dụng `COUNT ( 1 )` thay cho metric tổng diện tích vẫn tạo ra SQL hợp lệ, nhưng kết quả không trả lời đúng câu hỏi người dùng.

Sau kiểm tra nghiệp vụ, SQL được phân tích thành cây cú pháp trừu tượng AST. Guardrail kiểm tra loại statement, schema, table, column, hàm, phép aggregate, đường join và giới hạn số dòng.

Nếu vượt qua AST Guardrail, Core tiếp tục chạy:

EXPLAIN (FORMAT JSON)

để kiểm tra chi phí ước lượng trước khi thực thi thật. Truy vấn có chi phí vượt ngưỡng sẽ không được chạy mà được chuyển sang trạng thái yêu cầu người dùng thu hẹp phạm vi.

Nhờ vậy, giai đoạn xác minh kiểm soát đồng thời ba loại rủi ro:

Sai nghiệp vụ → SQL không an toàn → Query quá tốn tài nguyên

4. Giai đoạn thực thi an toàn – Safe Execution

Chỉ khi vượt qua toàn bộ các tầng kiểm tra, câu SQL mới được phép thực thi trên cơ sở dữ liệu.

Giai đoạn này gồm:

10. thực thi câu `SELECT` bằng role `assistant_ro`;
11. ghi audit và gắn metadata cho kết quả.

Tài khoản `assistant_ro` chỉ có quyền `SELECT` trên schema `assistant`. Nó không được cấp quyền trực tiếp trên các bảng nghiệp vụ trong schema `public`. Session truy vấn cũng được đặt ở chế độ `read-only` và có giới hạn thời gian thực thi.

Thiết kế này cung cấp một lớp bảo vệ ở cấp database. Ngay cả khi một lỗi bất thường vượt qua các tầng phía trên, quyền của database vẫn giới hạn hành vi của truy vấn.

Sau khi query hoàn thành, hệ thống ghi audit với các thông tin như:

-
- câu hỏi người dùng;
 - caller và user identity;
 - SQL trước và sau guardrail;
 - chi phí ước lượng;
 - số dòng trả về;
 - thời gian thực thi;
 - trạng thái thành công hoặc lỗi.

Đồng thời, Core gán các metadata như `metric_id` và đơn vị vào cột kết quả. Nhờ đó, phần diễn giải phía sau không cần tự suy đoán đơn vị của số liệu.

5. Giai đoạn trả kết quả – Result Delivery

Giai đoạn cuối cùng chịu trách nhiệm chuyển dữ liệu đã được xác minh về phía người dùng.

Ba bước cuối của pipeline gồm:

12. phát event `data_preview` về frontend;
13. LLM Synthesis diễn giải số liệu;
14. phát `data_answer` và kết thúc lượt xử lý.

Ngay sau khi database trả kết quả, Core không cần chờ LLM diễn giải xong mới hiển thị dữ liệu. Bảng kết quả được gửi sớm thông qua event:

`data_preview`

Điều này giúp giảm độ trễ cảm nhận của người dùng.

Sau đó, LLM ở tác vụ `synthesis` nhận một context có giới hạn gồm câu hỏi gốc, tên các cột, một phần dữ liệu, tổng số dòng thực tế và metadata đơn vị. Nhiệm vụ của LLM chỉ là diễn giải kết quả thành ngôn ngữ tự nhiên.

Cần nhấn mạnh rằng:

LLM không tạo ra số liệu cuối cùng.

Số liệu được lấy trực tiếp từ kết quả query đã được thực thi. LLM chỉ thực hiện chức năng trình bày và giải thích.

Cuối cùng, Core phát `data_answer`, lưu kết quả cần thiết phía server và kết thúc stream.

Từ năm giai đoạn trên, có thể mô hình hóa toàn bộ US03 theo chuỗi:

Hiểu truy vấn → Lập kế hoạch → Xác minh → Thực thi an toàn → Trả kết quả

Cách tổ chức này tạo ra ranh giới trách nhiệm rõ ràng giữa từng thành phần, đồng thời giúp việc kiểm thử, audit và truy vết lỗi có thể thực hiện ở từng giai đoạn thay vì chỉ dựa vào kết quả cuối cùng.

6.3.3 Vai trò của LLM và các thành phần tất định

Một đặc điểm trung tâm của US03 là LLM không kiểm soát toàn bộ pipeline. Hệ thống phân chia rõ những nhiệm vụ cần khả năng suy luận linh hoạt cho LLM và những quyết định phải được thực hiện bằng các thành phần tất định.

Trong một lượt xử lý thông thường, LLM tham gia tại bốn vị trí chính:

1. **Tool Selection:** phân tích ý định và đề xuất công cụ cần sử dụng;
2. **Object Selection:** lựa chọn Semantic View liên quan;
3. **SQL Generation:** sinh SQL Plan;
4. **Synthesis:** diễn giải kết quả số liệu.

Ngược lại, các thành phần tất định của Assistant Core đảm nhiệm:

- nhận diện và khóa required metric;
- bắt buộc source của metric;
- kiểm tra view tồn tại trong runtime schema;
- kiểm tra metric, measure, output và dimension;
- phân tích SQL bằng AST;
- phát hiện join có nguy cơ fan-out;
- áp dụng giới hạn số dòng;

- thực hiện EXPLAIN;
- kiểm tra ngưỡng chi phí;
- thực thi bằng role read-only;
- gán metadata;
- ghi audit.

Sự phân chia này được tổng hợp trong Bảng 8.

Bảng 8: Vai trò của LLM và thành phần tắt định trong US03

Giai đoạn	LLM	Assistant Core
Hiểu truy vấn	Phân tích ý định và đề xuất tool	Routing guardrail, nhận diện required metric
Lập kế hoạch	Lựa chọn view và sinh SqlPlan	Ép metric source, kiểm tra và giới hạn object
Xác minh	Có thể sinh lại SQL khi nhận feedback	Contract Validation, AST Guardrail và EXPLAIN
Thực thi	Không truy cập trực tiếp database	Thực thi bằng assistant_ro, timeout và audit
Trả kết quả	Diễn giải dữ liệu bằng ngôn ngữ tự nhiên	Metadata, data_preview, lưu và stream kết quả

Nguyên tắc thiết kế của US03 có thể được khái quát bằng ba bước:

LLM proposes – Core verifies – Database enforces

Trong đó, LLM có nhiệm vụ *đề xuất*; Assistant Core thực hiện *xác minh*; còn cơ sở dữ liệu thực hiện *cưỡng chế quyền truy cập*.

Ví dụ, prompt có thể yêu cầu LLM chỉ tạo câu SELECT, nhưng hệ thống không coi chỉ thị trong prompt là một biện pháp bảo mật đầy đủ. AST Guardrail vẫn phải kiểm tra lại cấu trúc SQL. Tương tự, việc LLM khai báo một `metric_id` trong `SqlPlan` không có nghĩa truy vấn được tin tưởng; Core vẫn đối chiếu biểu thức thực tế với `measure` đã định nghĩa trong Metric Registry.

Cách tiếp cận này cho phép tận dụng ưu điểm của LLM trong việc hiểu ngôn ngữ tự nhiên nhưng vẫn giới hạn tính phi định định của mô hình tại những vị trí có thể ảnh hưởng tới dữ liệu nghiệp vụ.

Do đó, mục tiêu của kiến trúc US03 không đơn thuần là tạo ra một câu SQL có thể thực thi. Một truy vấn chỉ được xem là hợp lệ khi đồng thời thỏa mãn bốn điều kiện:

-
- Đúng về nghiệp vụ: sử dụng đúng metric, source và phạm vi dữ liệu;
 - An toàn về cấu trúc: không chứa thao tác hoặc đường join bị cấm;
 - An toàn về tài nguyên: chi phí và kích thước kết quả nằm trong giới hạn;
 - Có khả năng truy vết: quá trình xử lý và kết quả đều được ghi audit.

Đây là nền tảng để các phần tiếp theo đi sâu vào từng thành phần của pipeline, bắt đầu từ giai đoạn tiếp nhận và hiểu truy vấn của người dùng. “‘latex

6.4 Tiếp nhận và phân tích truy vấn

Sau khi kiến trúc tổng thể xác định US03 là luồng chuyên trách truy vấn số liệu thực tế, bước đầu tiên trong quá trình xử lý là tiếp nhận câu hỏi và phân tích ý định nghiệp vụ của người dùng. Giai đoạn này có vai trò đặc biệt quan trọng bởi các quyết định được đưa ra ở đây ảnh hưởng trực tiếp tới toàn bộ pipeline phía sau. Nếu một câu hỏi bị định tuyến sai hoặc metric bị nhận diện sai, SQL được sinh ở các bước tiếp theo dù hợp lệ về cú pháp vẫn có thể trả về kết quả không đúng nhu cầu thực tế.

Khác với việc để một LLM duy nhất tự hiểu câu hỏi và quyết định toàn bộ cách truy vấn dữ liệu, US03 sử dụng sự kết hợp giữa *LLM-based routing* và các cơ chế kiểm tra tất định trong Assistant Core. Mục tiêu là tận dụng khả năng hiểu ngôn ngữ tự nhiên của LLM nhưng vẫn đảm bảo các câu hỏi số liệu không bị bỏ sót hoặc chuyển nhầm sang luồng giải thích và hướng dẫn.

Có thể mô tả giai đoạn tiếp nhận và phân tích truy vấn theo chuỗi sau:

User Question → Tool Routing → query_data → Metric Recognition →
Business Context

Trong đó, bước định tuyến xác định truy vấn có thuộc US03 hay không, còn bước nhận diện metric làm nhiệm vụ xác định đại lượng nghiệp vụ mà người dùng thực sự muốn lấy từ dữ liệu.

6.4.1 Định tuyến truy vấn vào *query_data*

Mọi câu hỏi gửi tới Assistant Core trước hết đều đi qua tầng điều phối trung tâm. Tại đây, một lời gọi LLM thuộc tác vụ Task.LIGHT được sử dụng để lựa chọn một hoặc nhiều công cụ phù hợp trong ba công cụ chính:

- search_docs: phục vụ US01, dùng cho các câu hỏi hướng dẫn thao tác;

-
- `lookup_field`: phục vụ US02, dùng cho các câu hỏi giải thích ý nghĩa, công thức hoặc quy tắc nghiệp vụ;
 - `query_data`: phục vụ US03, dùng cho các câu hỏi yêu cầu truy vấn số liệu thực tế.

Tool selector chỉ thực hiện nhiệm vụ phân loại ý định và không trực tiếp tạo câu trả lời cho người dùng. Việc tách riêng bước định tuyến khỏi bước xử lý nội dung giúp tránh trường hợp một LLM vừa lựa chọn nguồn dữ liệu vừa tự sinh câu trả lời mà không đi qua các cơ chế kiểm soát của từng luồng.

Đối với US03, chỉ dựa vào kết quả phân loại của LLM là chưa đủ. Do LLM mang bản chất xác suất, một câu hỏi có chứa cả thuật ngữ nghiệp vụ và yêu cầu định lượng có thể bị nhầm sang US02. Vì vậy, sau bước lựa chọn của LLM, Assistant Core áp dụng thêm một rào chắn tất định dành cho các câu hỏi số liệu.

Các dấu hiệu mạnh cho truy vấn US03 bao gồm những cụm từ như:

- “*bao nhiêu*”;
- “*có mấy*”;
- “*tổng*”;
- “*đếm*”;
- “*thống kê*”;
- “*liệt kê*”;
- “*tìm đơn*”.

Khi câu hỏi chứa các tín hiệu trên và công cụ `query_data` đang ở trạng thái sẵn sàng, Core bảo đảm US03 được đưa vào tập công cụ xử lý. Cơ chế này giúp tránh trường hợp câu hỏi số liệu bị đẩy hoàn toàn sang US01 hoặc US02 chỉ vì trong câu có xuất hiện tên trường, nhãn hoặc thuật ngữ nghiệp vụ.

Ví dụ, xét hai câu hỏi:

“*Tổng tiền đơn kéo là bao nhiêu?*”

và

“*Tổng tiền đơn kéo được tính như thế nào?*”

Mặc dù cùng đề cập tới khái niệm “tổng tiền”, hai câu hỏi có bản chất khác nhau. Câu thứ nhất yêu cầu một giá trị thực tế, do đó phải kích hoạt `query_data`. Câu thứ hai yêu cầu giải thích công thức, vì vậy thuộc phạm vi của US02.

Điểm phân biệt nằm ở ý định của người dùng:

Hỏi giá trị → US03

Hỏi cách tính → US02

Hệ thống cũng hỗ trợ trường hợp một câu hỏi chứa nhiều ý định đồng thời. Ví dụ:

“Tháng này có bao nhiêu đơn chờ duyệt và Chờ duyệt nghĩa là gì?”

Trong trường hợp này, Router không buộc câu hỏi phải thuộc duy nhất một US. Phần yêu cầu số lượng đơn được gửi tới `query_data`, trong khi phân giải thích trạng thái được gửi tới `lookup_field`. Hai công cụ có thể được thực thi song song, sau đó kết quả được tổng hợp ở tầng điều phối.

Một vấn đề khác trong quá trình định tuyến là các câu hỏi tiếp nối theo hội thoại. Người dùng có thể đặt câu hỏi đầy đủ ở lượt đầu:

“Tổng số đơn B2B tháng này là bao nhiêu?”

sau đó ở lượt tiếp theo chỉ hỏi:

“Còn tháng trước?”

Câu hỏi thứ hai không chứa đầy đủ đối tượng nghiệp vụ nhưng có cấu trúc rõ ràng của một truy vấn tiếp nối. Trong trường hợp này, Router có thể sử dụng lịch sử hội thoại để xây dựng lại ngữ cảnh cần thiết.

Tuy nhiên, hệ thống không tự động mang mọi điều kiện của câu hỏi trước sang câu hỏi sau. Nếu lượt hiện tại đã là một câu hỏi đầy đủ, các bộ lọc thời gian hoặc điều kiện của lượt trước không được tự ý áp dụng. Nguyên tắc này giúp hạn chế hiện tượng *context leakage*, trong đó một bộ lọc cũ vô tình làm thay đổi ý nghĩa của một truy vấn mới.

Có thể khái quát cơ chế định tuyến US03 thành hai lớp:

LLM Tool Selection → Deterministic Data-Query Guardrail

Lớp đầu tiên cung cấp tính linh hoạt để hiểu các cách diễn đạt tự nhiên khác nhau, trong khi lớp thứ hai bảo đảm các tín hiệu số liệu mạnh luôn được xử lý bởi đúng luồng.

Thiết kế này thể hiện nguyên tắc chung của Assistant Core:

Không sử dụng LLM như nguồn quyết định duy nhất đối với các bước ảnh hưởng tới dữ liệu.

LLM đề xuất hướng xử lý, nhưng Core vẫn áp dụng các luật tất định để đảm bảo truy vấn được đưa vào đúng pipeline.

6.4.2 Nhận diện Metric và ngữ cảnh nghiệp vụ

Sau khi câu hỏi đã được định tuyến vào `query_data`, nhiệm vụ tiếp theo của US03 là xác định đại lượng nghiệp vụ mà người dùng muốn truy vấn. Đây là bước *Metric Recognition*.

Trong hệ thống dữ liệu doanh nghiệp, một từ hoặc cụm từ ngôn ngữ tự nhiên thường không ánh xạ trực tiếp tới duy nhất một cột database. Ví dụ, cụm từ:

“tổng tiền”

có thể đại diện cho nhiều khái niệm khác nhau như:

- tổng tiền chưa VAT;
- tổng tiền có VAT;
- tiền bán;
- tiền net;
- giá trị báo cáo.

Tương tự, khái niệm:

“số lượng”

có thể biểu diễn:

- số đơn;
- số dòng dữ liệu;

-
- số lượng theo đơn vị tính;
 - diện tích m^2 ;
 - số pallet;
 - khối lượng kg.

Nếu SQL Generator được phép tự suy luận metric trực tiếp từ tên cột, hệ thống có nguy cơ chọn một đại lượng gần nghĩa nhưng không đúng với định nghĩa nghiệp vụ. Vì vậy, trước khi gọi LLM sinh SQL, US03 thực hiện một bước nhận diện metric bằng code dựa trên Metric Registry.

Quá trình có thể biểu diễn như sau:

Question → Normalize → Alias Matching → Unit Hint → Required Metric

Đầu tiên, câu hỏi được chuẩn hóa để giảm ảnh hưởng của các biến thể cách viết. Sau đó hệ thống so sánh nội dung câu hỏi với tập alias của các metric đã được phê duyệt.

Một nguyên tắc quan trọng là alias dài và cụ thể được ưu tiên hơn alias ngắn. Điều này giúp giảm xung đột giữa các metric có cách gọi gần giống nhau.

Ví dụ, nếu trong Metric Registry tồn tại hai alias:

- “*đơn kéo*”;
- “*đơn kéo hàng*”;

và người dùng hỏi:

“*Tổng m^2 đơn kéo hàng BNC Bắc Ninh tháng này.*”

thì alias “*đơn kéo hàng*” được ưu tiên vì mang ý nghĩa cụ thể hơn. Điều này giúp hệ thống lựa chọn metric thuộc phạm vi báo cáo thay vì một metric khác chỉ có tên gần giống.

Bên cạnh alias, các dấu hiệu về đơn vị cũng được sử dụng để thu hẹp tập metric. Một số dấu hiệu phổ biến gồm:

- m^2 , m^2 → metric diện tích;
- kg → metric khối lượng;
- pallet → metric theo pallet;

- VND, “*tiền*” → metric giá trị tiền tệ.

Sự kết hợp giữa alias và unit hint làm giảm đáng kể khả năng chọn nhầm metric.

Ví dụ:

“*Tổng m² đơn kéo hàng chi nhánh BNC Bắc Ninh tháng này.*”

Có thể được phân tích thành các thành phần nghiệp vụ như sau:

Bảng 9: Phân tích ngữ cảnh nghiệp vụ của một truy vấn US03

Thành phần	Giá trị nhận diện
Ý định	Tổng hợp số liệu thực tế
Đối tượng nghiệp vụ	Đơn kéo hàng
Đại lượng	Tổng diện tích
Đơn vị	m ²
Chi nhánh	BNC Bắc Ninh
Phạm vi thời gian	Tháng hiện tại
Luồng xử lý	query_data

Khi một metric được nhận diện thành công, metric đó được đưa vào tập:

`required_metrics`

Khái niệm “required” có ý nghĩa quan trọng trong pipeline. Nó cho biết đây không còn là một gợi ý dành cho LLM mà là một ràng buộc nghiệp vụ mà các bước sau phải tuân thủ.

Source của required metric sẽ được bắt buộc đưa vào tập Semantic View ở giai đoạn Object Selection. Khi LLM sinh SqlPlan, metric này cũng phải xuất hiện trong `metrics_used`. Ở bước Contract Validation, Core tiếp tục kiểm tra SQL thực tế có sử dụng đúng measure của metric hay không.

Nhờ đó, metric được duy trì xuyên suốt pipeline:

User Question → Required Metric → Source View → SqlPlan → Measure Validation → Result Metadata

Đây là một cơ chế quan trọng để bảo toàn ý nghĩa nghiệp vụ từ câu hỏi ban đầu tới kết quả cuối cùng.

Ngoài metric, US03 còn phải nhận diện *ngữ cảnh nghiệp vụ* của truy vấn. Ngữ cảnh này có thể bao gồm:

-
- phạm vi thời gian;
 - trạng thái;
 - chi nhánh;
 - module;
 - đơn vị tính;
 - đối tượng cần lọc;
 - các giá trị enum.

Tuy nhiên, hệ thống phải phân biệt giữa thông tin được người dùng yêu cầu và thông tin do mô hình tự suy luận. Ví dụ, nếu câu hỏi là:

“Tổng số đơn kéo hàng là bao nhiêu?”

thì SQL Generator không được tự động thêm điều kiện:

“tháng hiện tại”

chỉ vì nhiều truy vấn mẫu trước đó có sử dụng bộ lọc này.

Ngược lại, nếu câu hỏi là:

“Tổng số đơn kéo hàng tháng này là bao nhiêu?”

thì điều kiện thời gian phải được bảo toàn trong kế hoạch truy vấn.

Điều này thể hiện một nguyên tắc quan trọng:

US03 chỉ được làm rõ ý nghĩa đã tồn tại trong câu hỏi, không được tự mở rộng ý nghĩa của câu hỏi.

Trong trường hợp không nhận diện được metric, US03 không nhất thiết phải dừng ngay. Các truy vấn dạng danh sách hoặc phân tích trực tiếp theo cột vẫn có thể được xử lý nếu Semantic Layer cung cấp đầy đủ dữ liệu.

Ví dụ:

“Liệt kê các đơn B2B đang ở trạng thái chờ duyệt.”

không nhất thiết yêu cầu một metric tổng hợp. Hệ thống có thể lựa chọn Semantic View phù hợp và xây dựng một truy vấn dạng danh sách.

Tuy nhiên, nếu người dùng yêu cầu một phép tổng hợp chưa có metric được phê duyệt, việc LLM có khả năng viết một câu SUM không đồng nghĩa với việc phép tính đó được chấp nhận. Đây là ranh giới quan trọng giữa *khả năng sinh SQL* và *quyền định nghĩa nghiệp vụ*.

Có thể tóm tắt vai trò của giai đoạn tiếp nhận và phân tích truy vấn bằng hai câu hỏi:

1. Câu hỏi này có cần truy vấn dữ liệu thực tế hay không?
2. Nếu có, người dùng đang yêu cầu đại lượng nghiệp vụ nào và trong ngữ cảnh nào?

Chỉ sau khi hai câu hỏi trên được giải quyết, US03 mới chuyển sang giai đoạn lập kế hoạch truy vấn dữ liệu. Tại đó, required metric và ngữ cảnh vừa xác định sẽ được sử dụng để lựa chọn Semantic View và tạo SqlPlan. “

“‘latex

6.5 Lập kế hoạch truy vấn dữ liệu

Sau khi câu hỏi đã được định tuyến vào query_data và các metric bắt buộc đã được nhận diện, US03 chuyển sang giai đoạn lập kế hoạch truy vấn. Mục tiêu của giai đoạn này là xác định *nguồn dữ liệu nào cần được sử dụng và truy vấn cần được tổ chức như thế nào* trước khi tiến hành kiểm tra và thực thi.

Nếu cung cấp toàn bộ schema của hệ thống cho LLM ngay từ đầu, mô hình phải xử lý một không gian dữ liệu rất lớn, gồm nhiều view và hàng trăm cột. Điều này không chỉ làm tăng kích thước context mà còn làm tăng xác suất lựa chọn sai bảng, sai cột hoặc ghép nối các nguồn dữ liệu không liên quan. Vì vậy, US03 áp dụng chiến lược lập kế hoạch theo hai bước liên tiếp:

1. lựa chọn một tập nhỏ Semantic View có liên quan tới câu hỏi;
2. sử dụng tập view đã được kiểm soát để sinh một kế hoạch truy vấn có cấu trúc SqlPlan.

Có thể mô hình hóa giai đoạn này như sau:

Question + Required Metrics → Object Selection → Selected Semantic Views → Prompt Construction → SqlPlan

Cách tiếp cận này giúp tách riêng bài toán *chọn nguồn dữ liệu* khỏi bài toán *sinh SQL*. Nhờ đó, LLM ở bước SQL Generation chỉ được nhìn thấy một phần schema đã được thu hẹp thay vì toàn bộ không gian dữ liệu của hệ thống.

6.5.1 Lựa chọn Semantic View

Bước đầu tiên của quá trình lập kế hoạch là xác định các Semantic View có khả năng chứa dữ liệu cần thiết để trả lời câu hỏi. Thành phần chịu trách nhiệm cho bước này là một lời gọi LLM ở tác vụ Task.LIGHT.

Thay vì cung cấp toàn bộ cấu trúc chi tiết của từng view, hệ thống đưa cho LLM một catalog rút gọn mô tả các đối tượng dữ liệu đang có trong Semantic Layer. Nhiệm vụ của mô hình là lựa chọn một tập view có liên quan nhất tới câu hỏi hiện tại.

Kết quả lựa chọn không phải là văn bản tự do mà phải tuân theo một cấu trúc đầu ra xác định. Điều này giúp Assistant Core có thể kiểm tra và xử lý kết quả một cách tất định trước khi tiếp tục sang bước sinh SQL.

Trong thiết kế hiện tại, số lượng Semantic View được lựa chọn cho mỗi lượt được giới hạn tối đa ở một tập nhỏ. Việc áp dụng giới hạn này có hai mục tiêu chính:

- **Giảm kích thước context:** SQL Generator chỉ cần nhìn thấy schema của các view thực sự liên quan tới câu hỏi;
- **Giảm không gian lựa chọn:** càng ít bảng và cột được cung cấp, xác suất LLM sử dụng nhầm nguồn dữ liệu càng thấp.

Tuy nhiên, kết quả do LLM lựa chọn chưa được sử dụng trực tiếp. Assistant Core tiếp tục áp dụng các bước kiểm tra tất định nhằm bảo đảm tập view cuối cùng phù hợp với hợp đồng nghiệp vụ.

Quy trình có thể mô tả như sau:

LLM View Selection → Required Metric Source → Merge → Deduplicate → Runtime Validation

Trong đó, một bước quan trọng là bổ sung source của `required_metrics`. Nếu giai đoạn Metric Recognition trước đó đã xác định một metric bắt buộc, Semantic View được định nghĩa là nguồn của metric đó phải xuất hiện trong tập dữ liệu dùng cho SQL Generation.

Điều này có nghĩa rằng LLM không có quyền bỏ qua nguồn dữ liệu chính thức của metric. Ngay cả khi Object Selector không chọn view đó, Core vẫn tự động bổ sung nó vào tập kết quả.

Ví dụ, giả sử câu hỏi:

“Tổng m² đơn kéo hàng chi nhánh BNC Bắc Ninh tháng này.”

đã được nhận diện với một required metric tương ứng. Metric Registry xác định metric này phải được tính từ một Semantic View cụ thể. Khi đó:

Required Metric → Required Source View

là một ràng buộc bắt buộc, không phụ thuộc hoàn toàn vào quyết định của LLM.

Sau khi ghép lựa chọn của LLM với các nguồn bắt buộc, Core thực hiện loại bỏ các object trùng lặp và giới hạn lại số lượng view theo cấu hình của hệ thống. Tiếp theo, từng tên view được đối chiếu với schema assistant mà Core đã nội soi từ cơ sở dữ liệu khi runtime khởi động.

Nếu LLM đề xuất một view không tồn tại hoặc không nằm trong phạm vi runtime, hệ thống không tự động suy đoán một view khác có tên gần giống. Thay vào đó, luồng dừng với trạng thái phù hợp, tránh trường hợp mô hình “hallucinate” một đối tượng dữ liệu không thực sự tồn tại.

Có thể tóm tắt các trách nhiệm trong bước lựa chọn Semantic View như trong Bảng 10.

Bảng 10: Phân chia trách nhiệm trong bước lựa chọn Semantic View

Thành phần	Vai trò
LLM Object Selector	Đề xuất các Semantic View có liên quan tới câu hỏi dựa trên catalog rút gọn.
Metric Registry	Xác định source bắt buộc của các required metric.
Assistant Core	Ghép source bắt buộc, loại trùng, giới hạn số object và kiểm tra view với runtime schema.
Semantic Layer	Cung cấp tập view thực tế mà US03 được phép truy vấn.

Một lợi ích quan trọng của cơ chế này là thu hẹp không gian schema trước SQL Generation. Trong phạm vi hiện tại, Semantic Layer expose hàng trăm cột, nhưng không phải toàn bộ số cột này được đưa vào prompt của mọi câu hỏi. Thay vào đó, SQL Generator chỉ nhận schema chi tiết của các view đã được chọn.

Có thể mô tả sự thay đổi không gian tìm kiếm như sau:

Toàn bộ Semantic Schema → Object Selection → Một tập nhỏ Semantic Views → SQL Generation

Cách làm này vừa giúp giảm chi phí token, vừa giảm hiện tượng *schema confusion*, trong đó mô hình chọn nhầm cột chỉ vì nhiều bảng có các trường mang tên gần giống nhau.

Từ góc độ hệ thống, Object Selection đóng vai trò như một lớp *schema linking*. Nó tạo cầu nối giữa câu hỏi tự nhiên và không gian dữ liệu vật lý, nhưng vẫn giữ Assistant Core ở vị trí kiểm soát cuối cùng đối với các object được phép truyền sang SQL Generator.

6.5.2 Sinh kế hoạch truy vấn *SqlPlan*

Sau khi tập Semantic View đã được xác định và kiểm tra, Assistant Core chuyển sang bước xây dựng kế hoạch truy vấn. Đây là nơi LLM ở tác vụ `Task.SQL_GENERATION` được sử dụng để chuyển ngữ cảnh nghiệp vụ và schema thành một truy vấn SQL có cấu trúc.

Trước khi gọi mô hình, Core xây dựng một prompt chuyên biệt cho SQL Generation. Prompt không chỉ chứa câu hỏi người dùng mà còn kết hợp nhiều nguồn thông tin đã được kiểm soát từ các giai đoạn trước.

Các thành phần chính của prompt bao gồm:

1. các quy tắc SQL tĩnh của hệ thống;
2. các metric có liên quan, ưu tiên required metric;
3. schema chi tiết và grain của các Semantic View đã chọn;
4. thông tin về các đường join;
5. một số few-shot example có liên quan;
6. mapping các giá trị enum nếu có;
7. yêu cầu bắt buộc về metric;
8. ngày hiện tại;
9. câu hỏi người dùng;
10. thông tin lỗi từ lần sinh trước nếu hệ thống đang retry.

Có thể biểu diễn cấu trúc prompt theo các lớp:

SQL Rules
Metric Contract
Selected View Schema
Grain and Join Information
Few-shot Examples
Enum Mapping
Runtime Context
User Question
Previous Error (nếu có)

Thiết kế prompt theo cấu trúc trên giúp giới hạn “tầm nhìn” của SQL Generator. Mô hình không được tự do truy cập toàn bộ schema mà chỉ có thể xây dựng kế hoạch dựa trên những nguồn dữ liệu đã được Object Selection và Core phê duyệt.

Một đặc điểm quan trọng là SQL Generator không được yêu cầu trả về một câu SQL dạng text tự do. Thay vào đó, đầu ra phải tuân theo hợp đồng SqlPlan.

Có thể mô hình hóa SqlPlan như sau:

Thuộc tính	Ý nghĩa
status	Trạng thái của kế hoạch truy vấn
sql	Câu SQL đề xuất khi trạng thái hợp lệ
objects_used	Các Semantic View được sử dụng
metrics_used	Các metric được sử dụng trong truy vấn
assumptions	Các giả định mà mô hình đã áp dụng

Việc sử dụng structured output thay vì SQL text tự do mang lại hai lợi ích chính.

Thứ nhất, Core có thể kiểm tra không chỉ câu SQL mà còn kiểm tra các khai báo đi kèm của mô hình. Ví dụ, `metrics_used` có thể được đối chiếu với `required_metrics`, trong khi `objects_used` có thể được so sánh với tập Semantic View đã được phép.

Thứ hai, SqlPlan cho phép mô hình biểu diễn các trường hợp mà nó không đủ thông tin để tạo truy vấn hợp lệ thay vì buộc phải sinh một câu SQL bằng mọi giá.

Các trạng thái chính của SqlPlan gồm:

- **ok**: mô hình xác định đủ thông tin và cung cấp SQL;
- **need_clarification**: câu hỏi cần người dùng làm rõ;
- **out_of_scope**: yêu cầu nằm ngoài phạm vi dữ liệu được mở cho US03;
- **insufficient_schema**: ý nghĩa nghiệp vụ có thể hiểu được nhưng schema hiện tại không đủ để xây dựng truy vấn.

Ví dụ, nếu người dùng hỏi một metric nhưng không xác định rõ đối tượng hoặc một enum có thể được hiểu theo nhiều cách, SQL Generator không bắt buộc phải tự chọn một khả năng. Thay vào đó, nó có thể trả:

`need_clarification`

để yêu cầu thêm thông tin.

Tương tự, nếu câu hỏi đề cập tới một loại dữ liệu hoàn toàn không được expose trong Semantic Layer, trạng thái phù hợp là:

out_of_scope

thay vì tạo ra một bảng hoặc cột không tồn tại.

Cần nhấn mạnh rằng:

SqlPlan không phải là câu lệnh được phép thực thi.

Nó chỉ là *kế hoạch truy vấn do LLM đề xuất*. Ngay cả khi status = ok, câu SQL vẫn phải tiếp tục đi qua các lớp kiểm tra phía sau.

Có thể mô tả ranh giới này như sau:

LLM → SqlPlan → Chưa được thực thi

Sau đó:

SqlPlan → Business Contract Validation → SQL Guardrail → EXPLAIN → Execution

Đây là một khác biệt quan trọng giữa kiến trúc US03 và các hệ thống Text-to-SQL đơn giản. Trong các hệ thống thông thường, SQL do LLM tạo có thể được xem gần như là đầu ra cuối cùng của bước suy luận. Trong US03, SQL chỉ là một *candidate query* và quyền quyết định cuối cùng vẫn thuộc về Assistant Core.

Ngoài ra, *assumptions* trong SqlPlan có vai trò hỗ trợ audit, debug và kiểm thử. Các giả định của mô hình được giữ lại trong hợp đồng nội bộ để đội vận hành có thể truy vết nguyên nhân nếu kết quả không đúng mong đợi. Tuy nhiên, việc mô hình khai báo *assumption* không làm cho *assumption* đó tự động trở thành hợp lệ; các ràng buộc nghiệp vụ vẫn phải được kiểm tra độc lập bằng code.

Toàn bộ giai đoạn lập kế hoạch có thể tóm tắt bằng chuỗi:

Required Metrics → Semantic View Selection → Schema Reduction → Prompt Construction → SqlPlan

Giai đoạn này đóng vai trò chuyển tiếp giữa phần *hiểu câu hỏi* và phần *xác minh truy vấn*. Đầu ra của nó chưa phải kết quả dữ liệu mà là một kế hoạch truy vấn có cấu trúc, đủ thông tin để các thành phần tất định của Assistant Core tiếp tục kiểm tra về tính đúng đắn nghiệp vụ và an toàn trước khi cho phép truy vấn tiếp cận cơ sở dữ liệu. “

“‘latex

6.6 Kiểm tra tính đúng đắn của truy vấn

Sau khi LLM sinh ra `SqlPlan`, hệ thống chưa cho phép truy vấn được thực thi ngay. Ở giai đoạn này, Assistant Core thực hiện một lớp kiểm tra có mục tiêu xác định liệu câu SQL được đề xuất có thực sự phù hợp với ý nghĩa nghiệp vụ của câu hỏi hay không.

Đây là một bước đặc biệt quan trọng trong US03, bởi một câu SQL có thể hoàn toàn hợp lệ về mặt cú pháp nhưng vẫn trả về kết quả sai. Ví dụ, LLM có thể chọn đúng view nhưng sử dụng sai metric, dùng đúng metric nhưng lấy sai biểu thức measure, hoặc tự thêm một bộ lọc thời gian không tồn tại trong câu hỏi ban đầu. Những lỗi này thường không làm database báo lỗi, do đó không thể phát hiện chỉ bằng cách kiểm tra SQL syntax.

Vì vậy, US03 bổ sung một lớp *Business Contract Validation* nằm giữa SQL Generation và SQL Guardrail. Có thể biểu diễn vị trí của giai đoạn này trong pipeline như sau:

`SqlPlan` → `Business Contract Validation` → `SQL Guardrail` → `EXPLAIN` →
`Execution`

Mục tiêu của bước kiểm tra không phải là xác minh câu SQL có “chạy được” hay không, mà là xác minh rằng:

SQL có thực sự trả lời đúng câu hỏi người dùng hay không.

Nếu truy vấn vi phạm một ràng buộc nhưng lỗi có thể sửa, hệ thống không ngay lập tức kết thúc lượt xử lý. Thay vào đó, thông tin lỗi được phản hồi lại cho SQL Generator để mô hình tạo một `SqlPlan` mới. Cơ chế này hình thành một vòng *Self-Correction* có giới hạn.

6.6.1 Kiểm tra Hợp đồng nghiệp vụ

Hợp đồng nghiệp vụ là tập các ràng buộc được định nghĩa trong Binding Pack và Metric Registry nhằm bảo đảm rằng SQL do LLM sinh ra tuân thủ đúng ý nghĩa dữ liệu đã được phê duyệt.

Một nguyên tắc quan trọng của US03 là Assistant Core không tin hoàn toàn vào các trường mà LLM tự khai báo trong `SqlPlan`. Ví dụ, việc mô hình khai báo:

$$\text{metrics_used} = [\text{metric_A}]$$

không đồng nghĩa với việc SQL thực tế đã sử dụng đúng công thức của `metric_A`. Core phải tiếp tục kiểm tra trực tiếp cấu trúc của câu SQL.

Các nhóm kiểm tra chính được trình bày dưới đây.

1. Kiểm tra Object được sử dụng

Đầu tiên, hệ thống đối chiếu danh sách `objects_used` trong `SqlPlan` với tập `Semantic View` đã được lựa chọn ở bước trước.

Một object chỉ hợp lệ khi:

- tồn tại trong `schema assistant`;
- thuộc tập view được phép;
- nằm trong phạm vi đã được lựa chọn cho câu hỏi hiện tại.

Nếu LLM sinh ra một câu SQL tham chiếu tới một bảng ngoài phạm vi, truy vấn không được chấp nhận ngay cả khi bảng đó thực sự tồn tại trong database.

Có thể biểu diễn ràng buộc này như sau:

$$\text{objects_used} \subseteq \text{allowed semantic views}$$

Cơ chế này ngăn LLM mở rộng không gian dữ liệu ngoài phạm vi mà Object Selection đã xác định.

2. Kiểm tra Metric và Source

Nếu câu hỏi đã được nhận diện một hoặc nhiều `required_metrics`, các metric này bắt buộc phải xuất hiện trong kế hoạch truy vấn.

Ví dụ:

$$\text{required_metrics} = \{\text{pull_value_report_total_m2}\}$$

thì `SqlPlan` không được phép sử dụng một metric khác chỉ vì có ý nghĩa gần giống.

Hệ thống kiểm tra:

- metric được khai báo trong `metrics_used` có tồn tại trong Metric Registry hay không;
- required metric có thực sự xuất hiện hay không;
- metric có được truy vấn từ đúng source đã định nghĩa hay không.

Ràng buộc source đặc biệt quan trọng trong trường hợp cùng một đại lượng có thể xuất hiện ở nhiều view nhưng với grain hoặc phạm vi nghiệp vụ khác nhau.

Ví dụ, một giá trị “tổng m²” ở view báo cáo không nhất thiết tương đương với tổng m² được cộng trực tiếp từ danh sách đơn. Nếu metric quy định source cụ thể, LLM không được tự chuyển sang một nguồn gần nghĩa khác.

3. Kiểm tra biểu thức Measure

Một trong những kiểm tra quan trọng nhất là xác minh SQL có sử dụng đúng biểu thức measure của metric hay không.

Giả sử một metric được định nghĩa với measure:

```
1 SUM(effective_received_quantity)
```

nhưng LLM lại sinh:

```
1 COUNT(1)
```

Câu SQL vẫn có thể chạy thành công nhưng trả lời một đại lượng hoàn toàn khác.

Do đó, Core không chỉ đọc `metrics_used`, mà kiểm tra trực tiếp AST của biểu thức SQL để xác định measure có đúng với hợp đồng metric hay không.

Có thể mô tả yêu cầu này như:

Generated SQL Measure = Approved Metric Measure

Việc kiểm tra ở mức AST thay vì so sánh chuỗi giúp hệ thống đánh giá cấu trúc biểu thức SQL một cách đáng tin cậy hơn.

4. Kiểm tra Output Column

Metric Registry còn quy định tên cột đầu ra bắt buộc thông qua thuộc tính `output_column`.

Ví dụ, nếu metric yêu cầu:

`output_column = total_m2`

thì SQL phải trả kết quả với alias tương ứng.

Ràng buộc này không chỉ nhằm chuẩn hóa tên cột mà còn phục vụ quá trình xử lý kết quả phía sau. Result Metadata sử dụng tên output để ánh xạ đúng:

- `metric_id`;
- đơn vị;
- metadata của cột.

Nếu LLM tự đặt một alias tùy ý, Core có thể không xác định chính xác metadata nào cần gắn vào kết quả.

5. Kiểm tra Dimension

Một số metric không thể được diễn giải đầy đủ chỉ bằng giá trị số. Chúng yêu cầu một hoặc nhiều dimension đi kèm.

Ví dụ, nếu kết quả là:

100

nhưng đại lượng có thể được đo bằng m², kg hoặc pallet, giá trị này không đủ ý nghĩa nếu thiếu dimension hoặc đơn vị tương ứng.

Vì vậy, đối với các metric có khai báo `required_dimensions`, Core kiểm tra SQL có giữ lại đầy đủ các chiều bắt buộc và sử dụng đúng `dimension_aliases` hay không.

Điều này bảo đảm kết quả cuối cùng không bị mất ngữ cảnh nghiệp vụ.

6. Kiểm tra bộ lọc thời gian

Một lỗi thường gặp trong hệ thống sinh SQL bằng LLM là mô hình tự bổ sung một điều kiện mà người dùng không yêu cầu.

Ví dụ, người dùng hỏi:

“Tổng số đơn kéo hàng là bao nhiêu?”

nhưng SQL Generator tự thêm:

```
1 WHERE doc_date >= DATE_TRUNC('month', CURRENT_DATE)
```

khiến câu hỏi ban đầu bị thay đổi thành “tổng số đơn trong tháng hiện tại”.

Vì vậy, Contract Validation kiểm tra rằng SQL không tự áp dụng bộ lọc thời gian nếu câu hỏi không chứa yêu cầu tương ứng.

Ngược lại, nếu người dùng hỏi:

“Tổng số đơn kéo hàng tháng này là bao nhiêu?”

thì điều kiện thời gian lại là một phần bắt buộc của ý nghĩa truy vấn.

Nguyên tắc được áp dụng là:

Không tự thêm phạm vi thời gian ngoài câu hỏi người dùng.

7. Kiểm tra giá trị Enum và điều kiện lọc

Trong dữ liệu nghiệp vụ, một từ tiếng Việt có thể tương ứng với nhiều giá trị hoặc nhiều cột khác nhau.

Ví dụ, một khái niệm trạng thái có thể tồn tại ở nhiều trường trong cùng hệ thống. Nếu câu hỏi không đủ thông tin để xác định chính xác cột hoặc giá trị cần lọc, Core không cho phép LLM tự chọn một phương án tùy ý.

Trong trường hợp một giá trị enum mơ hồ và không thể phân giải an toàn bằng SQL, hệ thống chuyển sang trạng thái:

`need_clarification`

để yêu cầu người dùng cung cấp thêm thông tin.

Tổng hợp các kiểm tra nghiệp vụ có thể biểu diễn trong Bảng 11.

Bảng 11: Các kiểm tra chính trong Business Contract Validation

Thành phần	Nội dung kiểm tra	Rủi ro nếu không kiểm tra
Object	View có thuộc phạm vi được phép hay không	Truy vấn nhầm hoặc vượt phạm vi dữ liệu
Metric	Required metric có được sử dụng đúng hay không	Trả sai đại lượng nghiệp vụ
Source	Metric có lấy từ đúng Semantic View hay không	Dùng dữ liệu khác grain hoặc khác phạm vi
Measure	Biểu thức tính có đúng metric contract hay không	SQL chạy đúng nhưng kết quả sai
Output Column	Alias có đúng hợp đồng hay không	Không ánh xạ được metadata kết quả
Dimension	Có đủ chiều dữ liệu bắt buộc hay không	Kết quả mất ý nghĩa
Time Filter	Có đúng phạm vi thời gian trong câu hỏi hay không	Tự thay đổi ý nghĩa truy vấn
Enum	Giá trị lọc có được phân giải rõ ràng hay không	Lọc nhầm trạng thái hoặc đối tượng

Có thể thấy Business Contract Validation tạo ra một lớp phân biệt rõ giữa hai khái niệm:

Syntactically Correct SQL $\neq$ Business Correct SQL

Một truy vấn chỉ được chuyển tới SQL Guardrail khi đồng thời hợp lệ về mặt cấu trúc kế hoạch và ý nghĩa nghiệp vụ.

6.6.2 Cơ chế Self-Correction và Retry

Trong thực tế, LLM không phải lúc nào cũng sinh được SqlPlan chính xác ngay ở lần đầu tiên. Một lỗi có thể xuất hiện do mô hình lựa chọn sai alias, bỏ sót dimension, sử dụng sai measure hoặc tự thêm một bộ lọc không cần thiết.

Thay vì ngay lập tức kết thúc với trạng thái failed, US03 cho phép LLM tự sửa truy vấn dựa trên phản hồi của validator.

Cơ chế này được gọi là *Self-Correction*.

Luồng xử lý có thể biểu diễn như sau:

Generate SqlPlan $\rightarrow$ Validate $\rightarrow$

Pass	→ SQL Guardrail
Repairable Error	→ Regenerate
Ambiguous	→ need_clarification

Trong trường hợp lỗi có khả năng sửa, Assistant Core tạo một thông tin phản hồi mô tả nguyên nhân lỗi và bổ sung nó vào prompt của lần SQL Generation tiếp theo.

Ví dụ, lần sinh đầu tiên có thể tạo SQL:

```
1 SELECT COUNT(1) AS total_m2
2 FROM assistant.pull_order_report;
```

Trong khi metric contract yêu cầu sử dụng một biểu thức measure khác.

Validator có thể phát hiện lỗi:

“Metric được yêu cầu nhưng SQL không sử dụng đúng measure.”

Thông tin này được đưa lại cho SQL Generator cùng với context cũ. Mô hình sau đó có cơ hội sinh lại một kế hoạch phù hợp hơn.

Có thể mô tả vòng sửa như sau:

Question + Schema + Metric → SqlPlan₁ → Validation Error

↓

Question + Schema + Metric + Error Feedback → SqlPlan₂

Việc cung cấp lỗi cụ thể giúp LLM không phải suy đoán lại toàn bộ vấn đề từ đầu mà tập trung sửa đúng thành phần vi phạm hợp đồng.

Tuy nhiên, hệ thống không cho phép vòng Self-Correction chạy không giới hạn.

Trong thiết kế hiện tại, số lần retry được giới hạn ở:

tối đa 2 lần retry

sau lần sinh đầu tiên.

Như vậy, tổng số SqlPlan tối đa ở tầng nghiệp vụ là:

1 lần sinh ban đầu + 2 lần retry = 3 SqlPlan

Cơ chế này có thể được gọi là *Bounded Self-Correction*.

Việc giới hạn retry giải quyết ba vấn đề chính.

Thứ nhất, kiểm soát độ trễ. Mỗi lần retry yêu cầu thêm một lời gọi LLM. Nếu không có giới hạn, một truy vấn khó có thể tạo ra vòng lặp dài làm tăng thời gian phản hồi.

Thứ hai, kiểm soát chi phí. SQL Generation sử dụng token và tài nguyên của provider LLM. Giới hạn số lần sinh lại giúp hệ thống kiểm soát chi phí trên mỗi request.

Thứ ba, tránh vòng lặp không hội tụ. Có những trường hợp lỗi không đến từ cách LLM viết SQL mà do chính câu hỏi hoặc schema không đủ thông tin. Việc tiếp tục sinh lại SQL trong trường hợp này không làm vấn đề được giải quyết.

Do đó, hệ thống phân biệt giữa các lỗi có thể sửa và lỗi cần dừng.

Các trường hợp có thể retry thường bao gồm:

- bỏ sót required metric;
- sử dụng sai source;
- sai measure;
- sai output alias;
- thiếu dimension;
- SQL vi phạm một ràng buộc có thể sửa bằng cách thay đổi kế hoạch.

Ngược lại, hệ thống không nên tiếp tục retry khi:

- câu hỏi thực sự mơ hồ và cần người dùng xác nhận;
- dữ liệu nằm ngoài phạm vi Semantic Layer;
- schema hiện tại không đủ để biểu diễn yêu cầu;
- số lần retry đã đạt giới hạn.

Khi hết số lượt tự sửa mà vẫn không có SqlPlan hợp lệ, US03 trả về trạng thái:

`failed`

kèm nguyên nhân phù hợp thay vì tiếp tục cố gắng tạo ra một kết quả.

Một nguyên tắc đặc biệt quan trọng là khi Self-Correction thất bại, hệ thống không được biến lỗi thành một kết quả nghiệp vụ giả.

Ví dụ:

SQL không sinh được $\neq$ Result = 0

Giá trị 0 chỉ hợp lệ khi một câu SQL đúng đã thực thi thành công và dữ liệu thực tế cho kết quả bằng 0. Nếu pipeline thất bại, trạng thái lỗi phải được giữ nguyên.

Từ góc độ kiến trúc, Self-Correction tạo ra sự cân bằng giữa khả năng linh hoạt của LLM và tính kiểm soát của hệ thống:

LLM Flexibility + Deterministic Validation + Bounded Retry

LLM có cơ hội sửa những lỗi có thể phục hồi, nhưng Assistant Core vẫn quyết định lỗi nào được sửa, số lần sửa tối đa và thời điểm phải dừng.

Có thể tóm tắt toàn bộ giai đoạn kiểm tra tính đúng đắn bằng chuỗi:

SqlPlan $\rightarrow$ Business Validation $\rightarrow$

Valid	$\rightarrow$ SQL Safety Layer
Repairable	$\rightarrow$ Self-Correction
Ambiguous	$\rightarrow$ Clarification
Retry Exhausted	$\rightarrow$ Failed

Nhờ cơ chế này, US03 không phụ thuộc vào việc LLM phải tạo đúng SQL ngay từ lần đầu. Thay vào đó, tính chính xác được hình thành từ sự phối hợp giữa khả năng sinh kế hoạch của mô hình và lớp kiểm tra nghiệp vụ tất định của Assistant Core. Chỉ những truy vấn vượt qua giai đoạn này mới được chuyển tới tầng SQL Guardrail để tiếp tục kiểm tra về an toàn thực thi. “

“ ‘latex

6.7 Cơ chế bảo vệ và thực thi SQL

Sau khi SqlPlan vượt qua bước kiểm tra Hợp đồng nghiệp vụ, câu SQL vẫn chưa được phép thực thi trực tiếp trên cơ sở dữ liệu. US03 tiếp tục áp dụng một chuỗi các lớp bảo vệ kỹ thuật nhằm đảm bảo truy vấn vừa an toàn về cấu trúc, vừa không gây ảnh hưởng quá lớn tới tài nguyên hệ thống.

Các lớp kiểm soát chính gồm:

-
1. phân tích và kiểm tra SQL bằng AST Guardrail;
 2. kiểm soát phép tổng hợp và nguy cơ fan-out khi join;
 3. kiểm tra chi phí truy vấn bằng EXPLAIN;
 4. thực thi bằng tài khoản chỉ có quyền đọc.

Toàn bộ đường thực thi có thể mô hình hóa như sau:

Validated SQL → AST Guardrail → Aggregation/Fan-out Check → EXPLAIN
→ Read-only Execution

Điểm quan trọng trong thiết kế US03 là các lớp bảo vệ này độc lập với prompt của LLM. Việc prompt yêu cầu mô hình “chỉ tạo câu SELECT” không được xem là một cơ chế bảo mật đầy đủ. Mọi câu SQL sau khi được sinh ra đều phải được kiểm tra lại bằng code trước khi hệ thống cho phép kết nối tới database.

6.7.1 SQL Guardrail dựa trên AST

SQL Guardrail là lớp bảo vệ đầu tiên ở mức cấu trúc SQL. Thay vì kiểm tra truy vấn bằng cách tìm kiếm từ khóa trên chuỗi văn bản, hệ thống phân tích câu SQL thành cây cú pháp trừu tượng, hay *Abstract Syntax Tree – AST*.

Trong US03, SQL được parse theo cú pháp PostgreSQL. Sau khi parse thành công, Core có thể kiểm tra cấu trúc của truy vấn một cách chính xác hơn, chẳng hạn xác định loại statement, bảng được truy cập, các cột được sử dụng, phép join, hàm aggregate và các biểu thức con.

Có thể mô tả quá trình như sau:

Generated SQL → SQL Parser → AST → Guardrail Rules

AST Guardrail thực hiện một số nhóm kiểm tra quan trọng.

1. Chỉ cho phép truy vấn đọc dữ liệu

US03 chỉ được phép sinh và thực thi một câu truy vấn SELECT hoặc các phép set operation hợp lệ phục vụ việc đọc dữ liệu.

Các nhóm thao tác ghi hoặc thay đổi cấu trúc dữ liệu không được phép xuất hiện, chẳng hạn:

- INSERT;

- UPDATE;
- DELETE;
- DROP;
- ALTER;
- CREATE;
- các câu lệnh DDL hoặc DML khác.

Ngoài ra, hệ thống không cho phép nhiều statement trong cùng một request. Điều này ngăn trường hợp một câu SQL hợp lệ được ghép cùng một câu lệnh nguy hiểm phía sau.

Ví dụ, truy vấn:

```
1 SELECT *
2 FROM assistant.orders;
3
4 DROP TABLE assistant.orders;
```

sẽ bị từ chối vì chứa nhiều statement và có thao tác DDL.

2. Không cho phép **SELECT ***

Câu lệnh:

```
1 SELECT *
2 FROM assistant.some_view;
```

không được chấp nhận trong luồng US03.

Thay vào đó, SQL phải chỉ rõ các cột thực sự cần thiết. Quy định này giúp:

- giới hạn lượng dữ liệu đọc;
- tránh đưa ra các cột không cần thiết;
- giảm nguy cơ expose nhầm dữ liệu;
- giúp Core kiểm tra column-level access rõ ràng hơn.

Ví dụ:

```
1 SELECT
2     branch_code,
3     total_m2
4 FROM assistant.pull_order_report;
```

được kiểm soát tốt hơn so với việc lấy toàn bộ các cột trong view.

3. Kiểm soát Schema và Object

US03 chỉ được phép truy cập các object nằm trong schema ngữ nghĩa được phê duyệt, cụ thể là schema:

assistant

Do đó, truy vấn như:

```
1 SELECT order_id
2 FROM public.orders;
```

không được phép đi qua Guardrail.

Trong khi đó:

```
1 SELECT order_id
2 FROM assistant.orders;
```

chỉ được tiếp tục nếu `assistant.orders` nằm trong whitelist runtime của US03.

Cách làm này duy trì nguyên tắc:

Business Tables → Semantic Views → US03

thay vì:

US03 → Business Tables trực tiếp

Điều này giúp giữ ranh giới rõ ràng giữa tầng dữ liệu nghiệp vụ và tầng dữ liệu được thiết kế cho Assistant.

4. Kiểm tra Column

Không chỉ table/view mà từng cột xuất hiện trong câu SQL cũng được kiểm tra.

Nếu mô hình sinh ra một column không tồn tại trong runtime schema, truy vấn bị từ chối. Hệ thống không tự sửa bằng cách chọn một cột có tên gần giống.

Ví dụ, nếu Semantic View chỉ có:

```
1 branch_code
2 branch_name
3 total_m2
```

nhưng SQL sử dụng:

```
1 SELECT branch_id
2 FROM assistant.pull_order_report;
```

thì truy vấn không hợp lệ nếu `branch_id` không được expose trong view.

Cơ chế kiểm tra ở mức AST cũng giúp hệ thống phân biệt cột nằm trong:

- SELECT;
- WHERE;
- GROUP BY;
- ORDER BY;
- điều kiện JOIN.

Nhờ đó, một cột không hợp lệ không thể “ẩn” trong một thành phần khác của SQL.

5. Kiểm soát hàm SQL

Hệ thống chỉ cho phép các hàm nằm trong phạm vi an toàn đã định nghĩa. Những hàm không nằm trong danh sách được phép hoặc có khả năng tạo tác động ngoài phạm vi truy vấn dữ liệu có thể bị chặn.

Điều này tạo thêm một lớp giới hạn đối với SQL do LLM sinh ra, đặc biệt khi mô hình có thể sử dụng các hàm phức tạp không cần thiết.

6. Kiểm soát số dòng bằng LIMIT

Để hạn chế việc trả về một tập dữ liệu quá lớn, Guardrail áp dụng giới hạn số dòng kết quả.

Trong luồng hiện tại, câu SQL được bảo đảm có giới hạn tối đa theo cấu hình hệ thống. Nếu SQL không có LIMIT, Core có thể bổ sung giới hạn phù hợp; nếu SQL yêu cầu số dòng vượt mức cho phép, giới hạn được hạ xuống.

Có thể mô hình hóa nguyên tắc này như sau:

$$\text{Requested LIMIT} \leq \text{System Row Cap}$$

Nhờ đó, một yêu cầu quá rộng không thể khiến US03 trả về số lượng dòng không giới hạn.

Tóm lại, AST Guardrail không quyết định câu SQL có đúng nghiệp vụ hay không; nhiệm vụ đó đã được Business Contract Validation thực hiện ở bước trước. Guardrail tập trung vào câu hỏi khác:

“Cấu trúc SQL này có an toàn và nằm trong phạm vi kỹ thuật cho phép hay không?”

6.7.2 Kiểm soát Fan-out và phép tổng hợp

Một trong những lỗi nguy hiểm nhất trong truy vấn dữ liệu là *fan-out join*. Đây là trường hợp một bản ghi ở bảng hoặc view chính được ghép với nhiều bản ghi ở bảng khác, khiến các giá trị số bị lặp lại trước khi thực hiện phép tổng hợp.

Ví dụ, giả sử dữ liệu đơn hàng có:

$$\text{Order A} = 100 \text{ m}^2$$

và Order A có ba dòng chi tiết.

Nếu truy vấn thực hiện join:

```
1 SELECT SUM(o.total_m2)
2 FROM assistant.orders o
3 JOIN assistant.order_details d
4   ON o.order_id = d.order_id;
```

sau phép join, giá trị 100 m² có thể xuất hiện ba lần:

$$100 + 100 + 100 = 300$$

trong khi giá trị thực tế chỉ là:

$$100$$

Câu SQL trên hoàn toàn hợp lệ về mặt cú pháp và database vẫn trả kết quả thành công. Tuy nhiên, kết quả bị sai do thay đổi grain của dữ liệu.

Vì vậy, US03 sử dụng thông tin về *join cardinality* và grain trong Binding Pack để phát hiện các đường join có nguy cơ làm nhân bản dữ liệu.

Các quan hệ phổ biến có thể gồm:

1 : 1, 1 : N, N : 1

Đối với truy vấn chỉ liệt kê dữ liệu, một phép join 1 : N có thể hợp lệ. Tuy nhiên, khi truy vấn thực hiện:

SUM, AVG, COUNT

hoặc một phép aggregate khác, đường join này phải được kiểm tra cẩn thận.

Do đó, hệ thống không chỉ hỏi:

“Join này có hợp lệ không?”

mà còn cần xét:

“Join này có an toàn đối với phép aggregate hiện tại hay không?”

Có thể mô hình hóa quá trình như sau:

Join Path + Aggregation + Grain → Fan-out Validation

Nếu đường join có thể nhân bản measure trước khi aggregate, câu SQL bị chặn hoặc phải được sinh lại theo một chiến lược an toàn hơn.

Ví dụ, thay vì:

```
1 SELECT SUM(o.total_m2)
2 FROM assistant.orders o
3 JOIN assistant.order_details d
4   ON o.order_id = d.order_id;
```

hệ thống có thể yêu cầu truy vấn trực tiếp từ Semantic View đã được thiết kế đúng grain:

```
1 SELECT SUM(total_m2) AS total_m2
2 FROM assistant.order_value_report;
```

Việc ưu tiên Semantic View đã được chuẩn hóa giúp giảm số phép join tùy ý và giảm nguy cơ fan-out.

Một nhóm lỗi khác liên quan tới aggregate là việc thực hiện phép tổng hợp trên tập dữ liệu đã bị giới hạn trước.

Ví dụ:

```
1 SELECT SUM(total_m2)
2 FROM (
3     SELECT total_m2
4     FROM assistant.orders
5     LIMIT 100
6 ) t;
```

Trong trường hợp này, phép tổng hợp chỉ được thực hiện trên 100 dòng đầu tiên chứ không phải toàn bộ dữ liệu phù hợp với câu hỏi.

Nếu người dùng hỏi tổng toàn bộ dữ liệu, kết quả này sẽ sai nhưng database vẫn trả về bình thường.

Do đó, Guardrail chặn các trường hợp aggregate được thực hiện trên một dataset đã bị truncate hoặc giới hạn không phù hợp.

Nguyên tắc được sử dụng là:

LIMIT chỉ dùng để giới hạn kết quả trả về, không được làm thay đổi tập dữ liệu dùng để tính metric.

Nhờ kiểm soát fan-out và aggregate, US03 bảo vệ một lớp lỗi mà các cơ chế syntax validation thông thường không phát hiện được.

6.7.3 Kiểm soát chi phí bằng *EXPLAIN*

Sau khi SQL vượt qua AST Guardrail và các kiểm tra về aggregate, truy vấn vẫn chưa được thực thi ngay. Hệ thống tiếp tục đánh giá chi phí dự kiến bằng PostgreSQL:

EXPLAIN (FORMAT JSON)

Điểm quan trọng là EXPLAIN được thực hiện trên chính câu SQL đã qua Guardrail. Nhờ vậy, Core đánh giá đúng truy vấn sẽ được thực thi sau đó, thay vì một phiên bản khác của câu SQL.

Có thể biểu diễn bước này như sau:

Guarded SQL → EXPLAIN (FORMAT JSON) → Estimated Cost

PostgreSQL trả về query plan cùng các thông tin dự kiến về chi phí xử lý. Core lấy giá trị Total Cost và so sánh với ngưỡng được cấu hình.

Trong thiết kế US03 hiện tại, nếu:

$\text{Total Cost} \geq 60000$

truy vấn không được thực thi ngay.

Thay vào đó, hệ thống có thể chuyển về trạng thái:

`need_clarification`

và yêu cầu người dùng thu hẹp phạm vi, chẳng hạn theo:

- tháng;
- quý;
- khoảng ngày;
- chi nhánh;
- một đối tượng nghiệp vụ cụ thể.

Ví dụ, câu hỏi:

“Liệt kê toàn bộ đơn hàng từ trước đến nay.”

có thể tạo ra một query quá rộng.

Hệ thống có thể yêu cầu người dùng làm rõ:

“Bạn muốn xem dữ liệu trong tháng, quý hoặc khoảng thời gian nào?”

Cơ chế này giúp bảo vệ database khỏi các truy vấn vô tình quét một lượng dữ liệu quá lớn.

EXPLAIN được sử dụng thay vì chạy thử query vì nó cho phép hệ thống đánh giá kế hoạch truy vấn trước khi thực hiện xử lý thực tế.

Do đó, cost guardrail tạo thêm một lớp bảo vệ:

SQL hợp lệ $\nrightarrow$ SQL nên được chạy ngay

Một câu SQL có thể đúng cú pháp, đúng nghiệp vụ và không vi phạm AST nhưng vẫn quá tốn tài nguyên. Cost Guardrail giải quyết chính trường hợp này.

Có thể tổng hợp ba lớp kiểm tra trước execution như sau:

Bảng 12: Các lớp kiểm tra SQL trước khi thực thi

Lớp kiểm tra	Mục tiêu	Ví dụ lỗi phát hiện
Business Contract	Đảm bảo đúng nghiệp vụ	Sai metric, sai measure, sai filter
AST Guardrail	Đảm bảo an toàn cấu trúc SQL	DDL/DML, sai schema, SELECT *, sai column
Fan-out/Aggregation	Đảm bảo đúng grain và phép tổng hợp	Join nhân bản measure, aggregate sau LIMIT
Cost Guardrail	Đảm bảo chi phí nằm trong giới hạn	Full scan quá rộng, query có cost lớn

6.7.4 Thực thi Read-only

Chỉ khi vượt qua toàn bộ các tầng kiểm tra, SQL mới được phép thực thi trên cơ sở dữ liệu SCP.

Thứ tự thực thi được cố định:

AST Guardrail $\rightarrow$ EXPLAIN $\rightarrow$ SELECT $\rightarrow$ Audit

US03 không thực thi SQL bằng tài khoản owner hoặc superuser. Thay vào đó, Core sử dụng role:

`assistant_ro`

trong đó hậu tố ro thể hiện *read-only*.

Role này chỉ được cấp quyền đọc trên schema:

`assistant`

và không có quyền truy cập trực tiếp vào schema nghiệp vụ `public`.

Mô hình phân quyền có thể biểu diễn như sau:

US03 Core $\rightarrow$ assistant_ro $\rightarrow$ assistant schema $\rightarrow$ Semantic Views

Nhờ đó, ngay cả khi một câu SQL bất thường vượt qua các lớp phía trên, database vẫn cung cấp một lớp bảo vệ cuối cùng thông qua quyền của tài khoản kết nối.

Phiên thực thi còn được cấu hình ở chế độ:

`READ ONLY`

và có giới hạn thời gian bằng:

`statement_timeout`

Trong thiết kế hiện tại, thời gian thực thi tối đa cho query US03 là:

20 giây

Nếu truy vấn vượt quá thời gian này, database dừng statement thay vì để query chiếm tài nguyên trong thời gian dài.

Ngoài ra, hệ thống áp dụng giới hạn số dòng kết quả tối đa:

60.000 dòng

Giới hạn này phục vụ hai mục đích:

- bảo vệ tài nguyên phía database và Assistant Core;
- tránh truyền một tập dữ liệu quá lớn về frontend hoặc LLM.

Như vậy, ngay cả khi query cost nằm dưới ngưỡng cho phép, runtime vẫn có thêm các giới hạn độc lập về thời gian và số lượng dòng.

Từ góc độ bảo mật, US03 sử dụng mô hình *defense-in-depth*. Không có một lớp duy nhất chịu trách nhiệm bảo vệ toàn bộ hệ thống.

Các lớp bảo vệ lần lượt gồm:

Business Contract ↓

AST Guardrail ↓

Fan-out / Aggregation Check ↓

EXPLAIN Cost Guardrail ↓

Read-only Role ↓

Timeout + Row Cap

Các tầng trên xử lý các loại rủi ro khác nhau. Contract Validation chống sai nghiệp vụ; AST Guardrail chống SQL không an toàn; Fan-out Check chống sai số do thay đổi grain; EXPLAIN chống query quá nặng; còn database permission bảo đảm truy vấn chỉ có thể đọc dữ liệu đã được expose.

Việc phân tách tài khoản truy cập database cũng giúp giảm quyền của Assistant Core. Tài khoản owner được sử dụng cho các nhiệm vụ triển khai, tạo view hoặc cấp quyền, trong khi runtime của chatbot chỉ sử dụng DSN dành cho `assistant_ro`. Vì vậy, quyền quản trị database không cần xuất hiện trong luồng xử lý câu hỏi của người dùng.

Sau khi câu SELECT được thực thi, hệ thống thu nhận kết quả cùng các thông tin như:

- số dòng trả về;
- trạng thái có bị cắt theo giới hạn hay không;
- thời gian thực thi;
- SQL thực tế đã chạy;
- chi phí ước lượng trước execution;
- lỗi runtime nếu có.

Các thông tin này được chuyển sang bước Audit và xử lý kết quả ở phần tiếp theo.

Có thể tóm tắt toàn bộ cơ chế bảo vệ và thực thi SQL của US03 bằng chuỗi sau:

Validated SqlPlan → AST Validation → Fan-out Validation → EXPLAIN →
assistant_ro → Read-only SELECT → Result

Nhờ cách tổ chức nhiều tầng này, US03 không đặt niềm tin trực tiếp vào câu SQL do LLM sinh ra. Mọi truy vấn đều phải vượt qua các lớp kiểm soát độc lập trước khi tiếp cận dữ liệu thật. Đây là một trong những cơ chế cốt lõi giúp hệ thống có thể sử dụng LLM trong bài toán truy vấn dữ liệu doanh nghiệp mà vẫn duy trì được tính an toàn, khả năng kiểm soát và khả năng truy vết. “